

6. 哈希表

哈希表原理及实现

哈希表核心原理

引言

首先，我需要先阐明一个初学者很容易犯的概念错误。

请问，哈希表和我们常说的 Map（键值映射）是不是同一个东西？不是。

这一点用 Java 来讲解就很清楚，Map 是一个 Java 接口，仅仅声明了若干个方法，并没有给出方法的具体实现：

```
interface Map<K, V> {  
    V get(K key);  
    void put(K key, V value);  
    V remove(K key);  
    // ...  
}
```

Map 接口本身只定义了键值映射的一系列操作，HashMap 这种数据结构根据自身特点实现了这些操作。还有其他数据结构也实现了这个接口，比如 TreeMap、LinkedHashMap 等等。

换句话说，你可以说 HashMap 的 get, put, remove 方法的复杂度都是 $O(1)$ 的，但你不能说 Map 接口的复杂度都是 $O(1)$ 。因为如果换成其他的实现类，比如底层用二叉树结构实现的 TreeMap，这些方法的复杂度就变成 $O(\log N)$ 了。

我为什么要强调这一点呢？主要是针对使用非 Java 语言的读者。

其他编程语言可能没有 Java 这么清晰的接口定义，所以很容易让读者把哈希表和 Map 键值对混为一谈，听到键值对操作，就认为其增删查改的复杂度一定是 $O(1)$ 。这是不对的，具体要看这个底层的数据结构是如何实现键值操作的。

那么这一章节我会带大家动手实现一个哈希表，探讨哈希表为什么能做到增删查改 $O(1)$ 复杂度，以及解决哈希冲突的两种办法。

哈希表的基本原理

哈希表可以理解为一个加强版的数组。

数组可以通过索引在 $O(1)$ 的时间复杂度内查找到对应元素，索引是一个非负整数。

哈希表是类似的，可以通过 key 在 $O(1)$ 的时间复杂度内查找到这个 key 对应的 value。key 的类型可以是数字、字符串等多种类型。

怎么做的？特别简单，哈希表的底层实现就是一个数组（我们不妨称之为 `table`）。它先把这个 `key` 通过一个哈希函数（我们不妨称之为 `hash`）转化成数组里面的索引，然后增删查改操作和数组基本相同：

```
// 哈希表伪码逻辑
class MyHashMap {

private:
    vector<void*> table;

public:
    // 增/改，复杂度 O(1)
    void put(auto key, auto value) {
        int index = hash(key);
        table[index] = value;
    }

    // 查，复杂度 O(1)
    auto get(auto key) {
        int index = hash(key);
        return table[index];
    }

    // 删，复杂度 O(1)
    void remove(auto key) {
        int index = hash(key);
        table[index] = nullptr;
    }

private:
    // 哈希函数，把 key 转化成 table 中的合法索引
    // 时间复杂度必须是 O(1)，才能保证上述方法的复杂度都是 O(1)
    int hash(auto key) {
        // ...
    }
};
```

具体实现上有不少细节需要处理，比如哈希函数的设计、哈希冲突的处理等等。但你只要明白了上面的核心原理，就已经成功了一半了，剩下的就是写代码了，这有何难呢？

下面我们来具体介绍一下上述增删查改过程中几个关键的概念和可能出现的问题。

几个关键概念及原理

key 是唯一的，value 可以重复

哈希表中，不可能出现两个相同的 `key`，而 `value` 是可以重复的。

明白了上面讲的原理应该很好理解，你直接类比数组就行了：

数组里面每个索引都是唯一的，不可能说你这个数组有两个索引 0。至于数组里面存什么元素，随便你，没人 care。

所以哈希表是一样的，key 的值不可能出现重复，而 value 的值可以随意。

哈希函数

哈希函数的作用是任意长度的输入 (key) 转化成固定长度的输出 (索引)。

你也看到了，增删查改的方法中都会用到哈希函数来计算索引，如果你设计的这个哈希函数复杂度是 $O(N)$ ，那么哈希表的增删查改性能就会退化成 $O(N)$ ，所以说这个函数的性能很关键。

这个函数还要保证的一点是，输入相同的 key，输出也必须要相同，这样才能保证哈希表的正确性。不能说现在你计算 $\text{hash}("123") = 5$ ，待会儿计算 $\text{hash}("123") = 6$ ，这样的话哈希表就废了。

那么哈希函数是如何把非整数类型的 key 转化成整数索引的？又是如何保证这个索引是合法的呢？

如何把 key 转化成整数

这个问题可以有很多种答案，不同的哈希函数设计会有不同的方法，我这里就结合 Java 语言说一个简单的办法。其他编程语言也是类似的，可以参考这个思路，查询相关的标准库文档。

任意 Java 对象都会有一个 `int hashCode()` 方法，在实现自定义的类时，如果不重写这个方法，那么它的默认返回值可以认为是该对象的内存地址。一个对象的内存地址显然是全局唯一的一个整数。

所以我们只要调用 key 的 `hashCode()` 方法就相当于把 key 转化成了一个整数，且这个整数是全局唯一的。

当然，这个方法也有一些问题，下面会讲解，但现在至少找到了一种把任意对象转化为整数的方法。

如何保证索引合法

`hashCode` 方法返回的是 `int` 类型，首先一个问题就是，这个 `int` 值可能是负数，而数组的索引是非负整数。

那么你肯定想这样写代码，把这个值转化成非负数：

```
int h = key.hashCode();
if (h < 0) h = -h;
```

但这样有问题，`int` 类型可以表示的最小值是 -2^{31} ，而最大值是 $2^{31} - 1$ 。所以如果 $h = -2^{31}$ ，那么 $-h = 2^{31}$ 就会超出 `int` 类型的最大值，这叫做整型溢出，编译器会报错，甚至产生不可预知的结果。

为什么 `int` 的最小值是 -2^{31} ，而最大值是 $2^{31} - 1$ ？这涉及计算机补码编码的原理，简单说，`int` 就是 32 个二进制位，其中最高位（最左边那位）是符号位，符号位是 0 时表示正数，是 1 时表示负数。

现在的问题是，我想保证 `h` 非负，但又不能用负号直接取反。那么一个简单直接的办法是利用这种补码编码的原理，直接把最高位的符号位变成 0，就可以保证 `h` 是非负数了：

```
int h = key.hashCode();
// 位运算，把最高位的符号位去掉
// 另外，位运算的运行速度也会比一般的算术运算快
// 所以你看标准库的源码，能用位运算的地方它都会优先使用位运算
h = h & 0x7fffffff;
// 这个 0x7fffffff 的二进制表示是 0111 1111 ... 1111
// 即除了最高位（符号位）是 0，其他位都是 1
// 把 0x7fffffff 和其他 int 进行 & 运算之后，最高位（符号位）就会被清零，即保证了 h
是非负数
```

关于补码编码的原理我这里就不详细展开了，有兴趣的话你可以自己搜索学习一下。

好的，上面解决了 `hashCode` 可能是负数的问题，但还有一个问题，就是这个 `hashCode` 一般都很大，我们需要把它映射成 `table` 数组的合法索引。

这个问题对你来说应该不难吧，我们之前在 [环形数组原理及实现](#) 里面用 `%` 求模运算来保证索引永远落在数组的合法范围内。所以这里也可以用 `%` 运算来保证索引的合法性，完整的 `hash` 函数实现如下：

```
int hash(K key) {
    int h = key.hashCode();
    // 保证非负数
    h = h & 0x7fffffff;
    // 映射到 table 数组的合法索引
    return h % table.length;
}
```

当然，直接使用 `%` 也有问题，因为 `%` 这个求余数的运算比较消耗性能，一般在追求运行效率的标准库源码中会尽量避免使用 `%` 运算，而是使用位运算提升性能。

不过本章主要目的是带你理解实现一个简单的哈希表，就忽略这些细节优化了。有兴趣的话你可以去看一下 Java `HashMap` 的源码，看看它是如何实现这个 `hash` 函数的。

哈希冲突

上面给出了 `hash` 函数的实现，那么你一定也会想到，如果两个不同的 `key` 通过哈希函数得到了相同的索引，怎么办呢？这种情况就叫做「哈希冲突」。

哈希冲突是否可以避免？

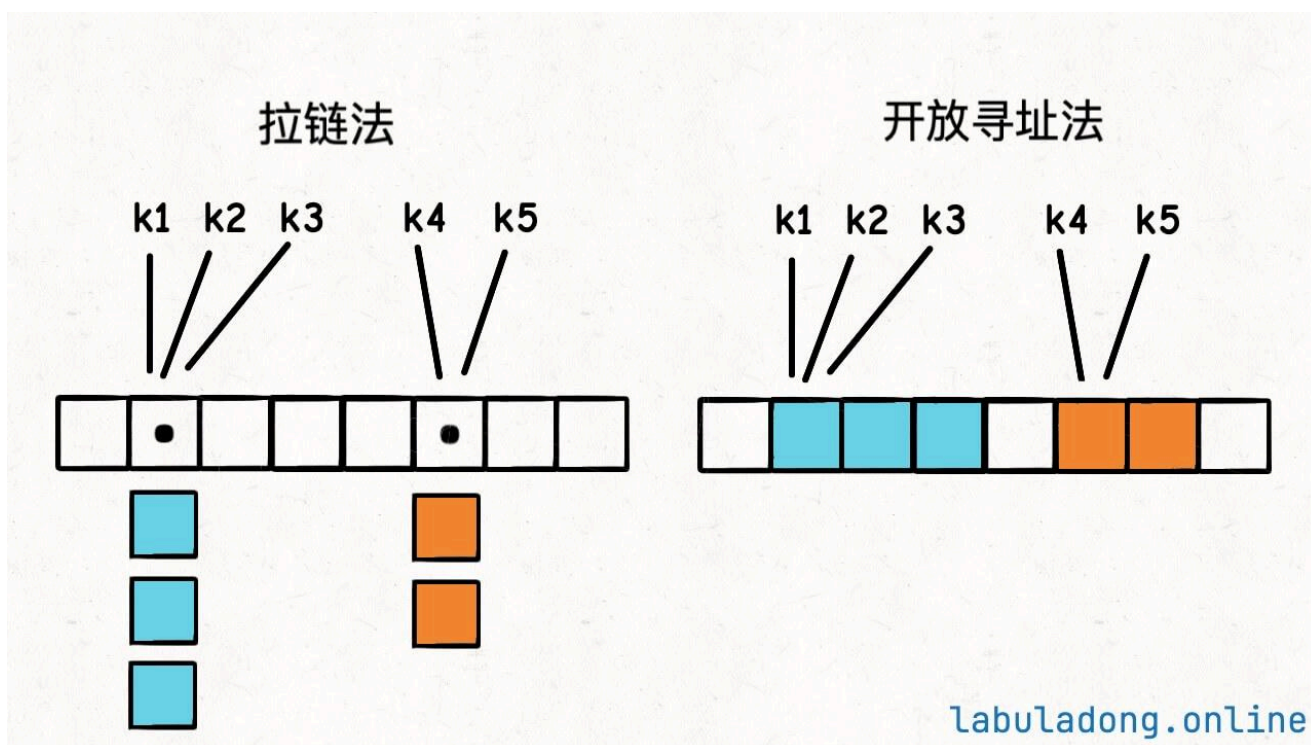
哈希冲突不可能避免，只能在算法层面妥善处理出现哈希冲突的情况。

哈希冲突是一定会出现的，因为这个 `hash` 函数相当于是把一个无穷大的空间映射到了一个有限的索引空间，所以必然会有不同的 `key` 映射到同一个索引上。

就好比三维物体映射到二维影子一样，这种有损压缩必然会出现信息丢失，有损信息本就无法和原信息一一对应。

出现哈希冲突的情况怎么解决？两种常见的解决方法，一种是**拉链法**，另一种是**线性探查法**（也经常被叫做**开放寻址法**）。

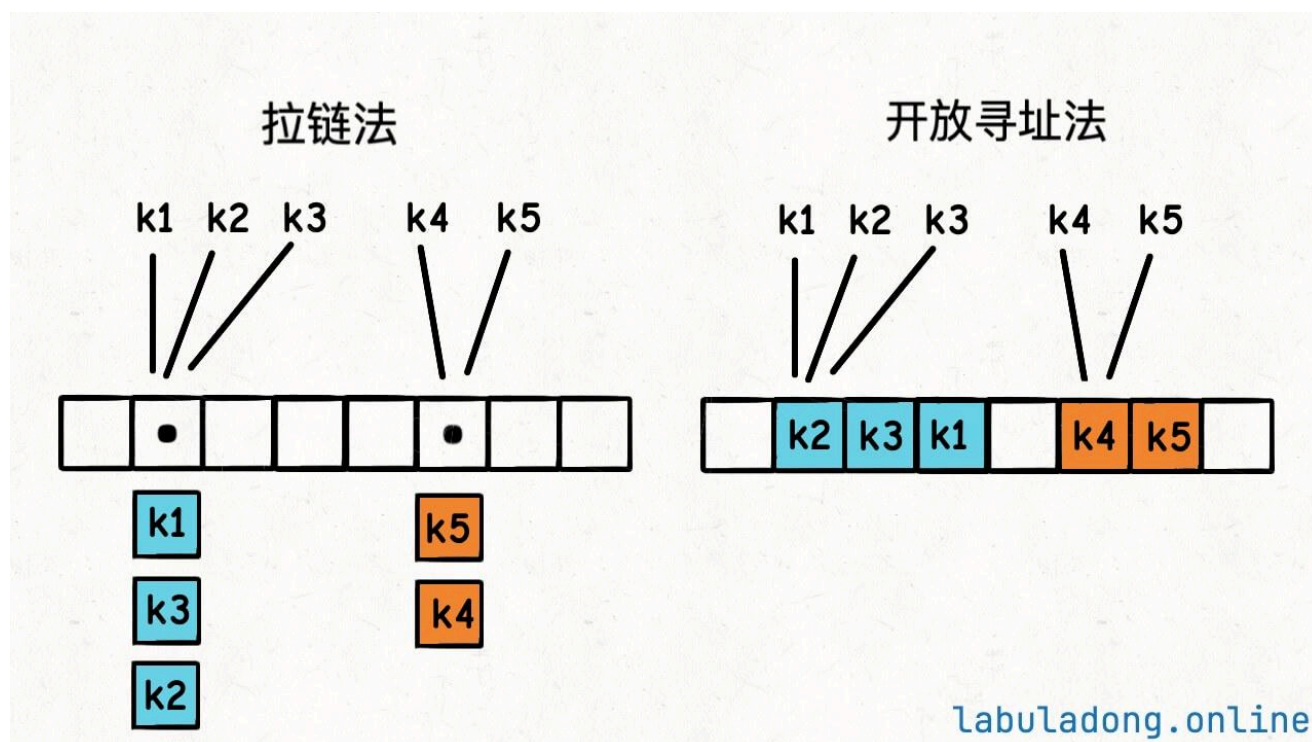
名字听起来高大上，说白了就是纵向延伸和横向延伸两种思路嘛：



拉链法相当于是哈希表的底层数组并不直接存储 `value` 类型，而是存储一个链表，当有多个不同的 `key` 映射到了同一个索引上，这些 `key -> value` 对儿就存储在这个链表中，这样就能解决哈希冲突的问题。

而线性探查法的思路是，一个 `key` 发现算出来的 `index` 值已经被别的 `key` 占了，那么它就去 `index + 1` 的位置看看，如果还是被占了，就继续往后找，直到找到一个空的位置为止。

比方说上图，`key` 的插入顺序是 `k2, k4, k5, k3, k1`，那么哈希表底层就会变成这样：



这里先讲一下原理，后面的章节我会手把手带大家分别实现这两种方法来解决哈希冲突。

扩容和负载因子

相信大家都听说过「负载因子」这个专业术语，现在你明白了哈希冲突的问题，就能理解负载因子的意义了。

拉链法和线性探查法虽然能解决哈希冲突的问题，但是它们会导致性能下降。

比如拉链法，你算出来 $\text{index} = \text{hash}(\text{key})$ 这个索引了，结果过去查出来的是个链表，你还得遍历一下这个链表，才能在里面找到你要的 value 。这个过程的时间复杂度是 $O(K)$ ， K 是这个链表的长度。

线性探查法也是类似的，你算出来 $\text{index} = \text{hash}(\text{key})$ 这个索引了，你去这个索引位置查看，发现存储的不是要找的 key ，但由于线性探查法解决哈希冲突的方式，你并不能确定这个 key 真的不存在，你必须顺着这个索引往后找，直到找到一个空的位置或者找到这个 key 为止，这个过程的时间复杂度也是 $O(K)$ ， K 为连续探查的次数。

所以说，如果频繁出现哈希冲突，那么 K 的值就会增大，这个哈希表的性能就会显著下降。这是我们需要避免的。

那么为什么会频繁出现哈希冲突呢？两个原因呗：

- 1、哈希函数设计的不好，导致 key 的哈希值分布不均匀，很多 key 映射到了同一个索引上。
- 2、哈希表里面已经装了太多的 key-value 对了，这种情况下即使哈希函数再完美，也没办法避免哈希冲突。

对于第一个问题没什么好说的，开发编程语言标准库的大佬们已经帮你设计好了哈希函数，你只要调用就行了。

对于第二个问题是我们可以控制的，即避免哈希表装太满，这就引出了「负载因子」的概念。

负载因子

负载因子是一个哈希表装满的程度的度量。一般来说，负载因子越大，说明哈希表里面存储的 key-value 对越多，哈希冲突的概率就越大，哈希表的操作性能就越差。

负载因子的计算公式也很简单，就是 `size / table.length`。其中 `size` 是哈希表里面的 key-value 对的数量，`table.length` 是哈希表底层数组的容量。

你不难发现，用拉链法实现的哈希表，负载因子可以无限大，因为链表可以无限延伸；用线性探查法实现的哈希表，负载因子不会超过 1。

像 Java 的 HashMap，允许我们创建哈希表时自定义负载因子，不设置的话默认是 0.75，这个值是经验值，一般保持默认就行了。

当哈希表内元素达到负载因子时，哈希表会扩容。和之前讲解 [动态数组的实现](#) 是类似的，就是把哈希表底层 `table` 数组的容量扩大，把数据搬移到新的大数组中。`size` 不变，`table.length` 增加，负载因子就减小了。

为什么不能依赖哈希表的遍历顺序

你大概也听过一个编程常识，即哈希表中键的遍历顺序是无序的，不能依赖哈希表的遍历顺序来编写程序。这是为什么呢？

哈希表的遍历本质上就是遍历那个底层 `table` 数组：

```
// 遍历所有 key 的伪码逻辑

// 哈希表底层的 table 数组
KVNode[] table = new KVNode[1000];

// 获取哈希表中的所有键
// 我们不能依赖这个 keys 列表的顺序
List<KeyType> keys = new ArrayList<>();

for (int i = 0; i < table.length; i++) {
    KVNode node = table[i];
    if (node != null) {
        keys.add(node.key);
    }
}
```

你如果理解了前面讲的内容，应该已经能够理解这个问题了。

首先，由于 `hash` 函数要把你的 `key` 进行映射，所以 `key` 在底层 `table` 数组中的分布是随机的，不像数组/链表结构那样有个明确的元素顺序。

其次，刚才讲了哈希表达到负载因子时会怎样？会扩容对吧，也就是 `table.length` 会变化，且会搬移元素。

那么这个搬移数据的过程，是不是要用 `hash` 函数重新计算 `key` 的哈希值，然后放到新的 `table` 数组中？

而这个 `hash` 函数，它计算出的索引值依赖 `table.length`。也就是说，哈希表自动扩缩容后，同一个 `key` 存储在 `table` 的索引可能发生变化，所以遍历结果的顺序就和之前不一样了。

你观察到的现象就是，这次遍历的第一个键是 `key1`，但是增删几个元素再遍历，可能发现 `key1` 跑到最后去了。

所以说，这些东西没必要背的，原理搞明白了，你稍微推理下自己都能想通。

为什么不建议在 `for` 循环中增/删哈希表的 `key`

注意我这里说的是不建议，并不是一定不可以。因为不同的编程语言标准库对哈希表的实现不同，有些语言针对这种情况做了优化，所以到底行不行，要查阅文档。

我们这里仅从哈希表的原理上分析，在 `for` 循环中增/删哈希表的 `key`，是很容易出现问题的，原因和上面相同，还是扩缩容导致的哈希值变化。

遍历哈希表的 `key`，本质就是遍历哈希表底层的 `table` 数组，如果一边遍历一边增删元素，如果遍历到一半，插入/删除操作触发了扩缩容，整个 `table` 数组都变了，那么请问，接下来应该是什么行为？还有，在遍历过程中新插入/删除的元素，是否应该被遍历到？

扩缩容导致 `key` 顺序变化是哈希表的特有行为，但即便排除这个因素，任何其他数据结构，也都不建议在遍历的过程中同时进行增删，否则很容易导致非预期的行为。

如果你非要这样做，请确保查阅了相关文档，明确这个操作的行为是什么，做到心里有数。

`key` 必须是不可变的

只有那些不可变类型，才能作为哈希表的 `key`，这一点很重要。

所谓不可变类型，就是说这个对象一旦创建，它的值就不能再改变了。比如 Java 中的 `String`、`Integer` 等类型，一旦创建了这些对象，你就只能读取它的值，而不能再修改它的值了。

作为对比，Java 中的 `ArrayList`、`LinkedList` 这些对象，它们创建出来之后，可以往里面随意增删元素，所以它们是可变类型。

因此，你可以把 `String` 对象作为哈希表的 `key`，但不能把 `ArrayList` 对象作为哈希表的 `key`：

```
// 可以把不可变类型作为 key
Map<String, AnyOtherType> map1 = new HashMap<>();
Map<Integer, AnyOtherType> map2 = new HashMap<>();
```



```
// 不应该把可变类型作为 key
// 注意，这样写并不会产生语法错误，但是代码非常容易出 bug
Map<ArrayList<Integer>, AnyOtherType> map3 = new HashMap<>();
```

为啥不建议把可变类型作为 key 呢？就比如这个 ArrayList 吧，它的 hashCode 方法的实现逻辑如下：

```
public int hashCode() {
    for (int i = 0; i < elementData.length; i++) {
        h = 31 * h + elementData[i];
    }
}
```

第一个就是效率问题，每次计算 hashCode 都要遍历整个数组，复杂度是 $O(N)$ ，这样就会导致哈希表的增删查改操作的复杂度退化到 $O(N)$ 。

更严重的问题是，ArrayList 的 hashCode 是根据它里面的元素计算出来的，如果你往这个 ArrayList 里面增删元素，或者其中某个元素的 hashCode 值发生改变，那么这个 ArrayList 的 hashCode 返回值也会发生改变。

比方说，你现在用一个 ArrayList 类型的 arr 变量作为哈希表的 key 在哈希表中保存了对应的 value。但如果 arr 中的某个元素在程序的其他位置被修改了，那么 arr 的 hashCode 就会变化。此时你再用这个 arr 变量去哈希表中查询，发现找不到任何值了。

也就是说，你存入哈希表的 key-value 意外丢失了，这是非常非常严重的 bug，还会带来潜在的内存泄漏问题。

```
public class Test {
    public static void main(String[] args) {
        // 错误示例
        // 把可变类型作为 HashMap 的 key
        Map<ArrayList<Integer>, Integer> map = new HashMap<>();

        ArrayList<Integer> arr = new ArrayList<>();
        arr.add(1);
        arr.add(2);

        map.put(arr, 999);
        System.out.println(map.containsKey(arr)); // true
        System.out.println(map.get(arr)); // 999

        arr.add(3);
        // 出现严重 bug，键值对丢失
        System.out.println(map.containsKey(arr)); // false
        System.out.println(map.get(arr)); // null
    }
}
```

```
// 此时 map 底层的 table 中, arr 的键值对数据依然存在
// 但是由于 arr 的 hashCode 改变了, 此键值对无法被查找到
// 这也会导致内存泄漏, 因为这个 arr 变量被 map 引用着, 无法被垃圾回收
    }
}
```

上面就是一个简单的错误示例。你也许会说, 把元素 3 删掉, `arr -> 999` 这个键值对不就又出现了? 或者, 直接遍历哈希表底层的 `table` 数组, 应该也可以看到这个键值对。

拜托🙏, 你这是在写代码还是在写盗墓笔记呢? 一会儿出现一会儿消失, 你这个哈希表是幽灵附体了吗?

开个玩笑。实际上可变类型本身就是一种不确定性, 在代码构成的屎山里, 你怎么知道这个 `arr` 传递到哪里被修改了呢?

所以正确的做法是, 使用不可变类型作为哈希表的 `key`, 比方说用 `String` 类型作为 `key`。因为 Java 中的 `String` 对象一旦创建出来, 它的值就不允许被改变, 你就不会遇到上面的问题。

`String` 类型的 `hashCode` 方法也需要遍历所有字符, 但是由于它的不可变性, 这个值只要算出来一次, 就可以缓存下来, 不用每次都重新计算, 所以 [平均时间复杂度](#) 依然是 $O(1)$ 。

我这里是用 Java 举的例子, 其他语言也是类似的, 你需要查询相关文档, 了解标准库提供的哈希表是如何计算对象哈希值的, 避免产生类似的问题。

总结

上面的说明应该已经把哈希表的底层原理全部串起来了, 最后模拟几个面试问题来总结一下本文的内容:

1、为什么我们常说, 哈希表的增删查改效率都是 $O(1)$?

因为哈希表底层就是操作一个数组, 其主要的时间复杂度来自于哈希函数计算索引和哈希冲突。只要保证哈希函数的复杂度在 $O(1)$, 且合理解决哈希冲突的问题, 那么增删查改的复杂度就都是 $O(1)$ 。

2、哈希表的遍历顺序为什么会变化?

因为哈希表在达到负载因子时会扩容, 这个扩容过程会导致哈希表底层的数组容量变化, 哈希函数计算出来的索引也会变化, 所以哈希表的遍历顺序也会变化。

3、哈希表的增删查改效率一定是 $O(1)$ 吗?

不一定, 正如前面分析的, 只有哈希函数的复杂度是 $O(1)$, 且合理解决哈希冲突的问题, 才能保证增删查改的复杂度是 $O(1)$ 。

哈希冲突好解决, 都是有标准答案的。关键是哈希函数的计算复杂度。如果使用了错误的 `key` 类型, 比如前面用 `ArrayList` 作为 `key` 的例子, 那么哈希表的复杂度就会退化

成 $O(N)$ 。

4、为啥一定要用不可变类型作为哈希表的 key？

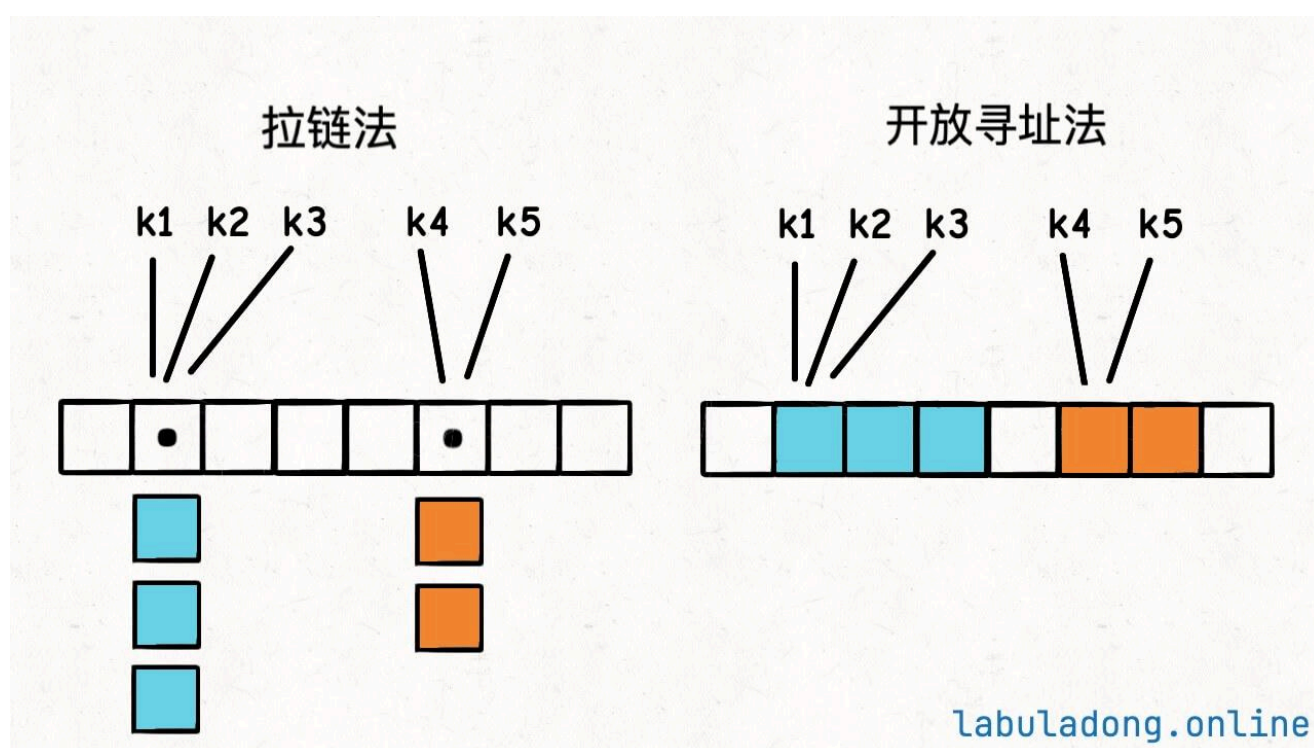
因为哈希表的主要操作都依赖于哈希函数计算出来的索引，如果 key 的哈希值会变化，会导致键值对意外丢失，产生严重的 bug。

要对自己使用的编程语言标准库中的源码有一定的了解，才能保证写出高效的代码。

下面，就我将手把手带大家分别用拉链法和线性探查法来实现简单的哈希表，来加深对哈希表的理解。

用拉链法实现哈希表

前文 [哈希表核心原理](#) 中我介绍了哈希表的核心原理和几个关键概念，其中提到了解决哈希冲突的方法主要有两种，分别是拉链法和开放寻址法（也常叫做线性探查法）：



本文就来具体介绍一下拉链法的实现原理和代码。

拉链法的简化版实现

[哈希表核心原理](#) 已经介绍过哈希函数和 key 的类型的关系，其中 hash 函数的作用是在 $O(1)$ 的时间把 key 转化成数组的索引，而 key 可以是任意不可变的类型。

但是这里为了方便诸位理解，我先做如下简化：

- 1、我们实现的哈希表只支持 key 类型为 int，value 类型为 int 的情况，如果 key 不存在，就返回 -1。
- 2、我们实现的 hash 函数就是简单地取模，即 $\text{hash}(\text{key}) = \text{key} \% \text{table.length}$ 。这样也方便模拟出哈希冲突的情况，比如当 $\text{table.length} = 10$ 时， $\text{hash}(1)$ 和 $\text{hash}(11)$ 的值

都是 1。

3、底层的 `table` 数组的大小在创建哈希表时就固定，不考虑负载因子和动态扩扩容的问题。

这些简化能够帮助我们聚焦增删查改的核心逻辑。

```
#include <vector>
#include <list>
#include <algorithm>

// 用拉链法解决哈希冲突的简化实现
class ExampleChainingHashMap {

    // 链表节点，存储 key-value 对儿
    // 注意这里必须存储同时存储 key 和 value
    // 因为要通过 key 找到对应的 value
    struct KVNode {
        int key;
        int value;

        // 为了简化，我这里直接用标准库的 LinkedList 链表
        // 所以这里就不添加 next 指针了
        // 你当然可以给 KVNode 添加 next 指针，自己实现链表操作
        KVNode(int key, int value) : key(key), value(value) {}
    };

    // 底层 table 数组中的每个元素是一个链表
    std::vector<std::list<KVNode>> table;

public:
    ExampleChainingHashMap(int capacity) : table(capacity) {}

    int hash(int key) {
        return key % table.size();
    }

    // 查
    int get(int key) {
        int index = hash(key);

        if (table[index].empty()) {
            // 链表为空，说明 key 不存在
            return -1;
        }

        for (const auto& node : table[index]) {
            if (node.key == key) {
                return node.value;
            }
        }
    }
};
```

```

    }
}

// 链表中没有目标 key
return -1;
}

// 增/改
void put(int key, int value) {
    int index = hash(key);

    if (table[index].empty()) {
        // 链表为空，新建一个链表，插入 key-value
        table[index].push_back(KVNode(key, value));
        return;
    }

    // 链表不为空，要遍历一遍看看 key 是否存在
    // 如果存在，更新 value
    // 如果不存在，插入新节点
    for (auto& node : table[index]) {
        if (node.key == key) {
            // key 已经存在，更新 value
            node.value = value;
            return;
        }
    }

    // 链表中没有目标 key，添加新节点
    // 这里使用 push_back 添加到链表尾部
    // 因为 c++ std::list 的底层实现是双链表，头尾操作都是 O(1) 的
    // https://labuladong.online/algorithm/data-structure-basic/linkedlist-
    implement/
    table[index].push_back(KVNode(key, value));
}

// 删
void remove(int key) {
    auto& list = table[hash(key)];
    if (list.empty()) {
        return;
    }

    // 如果 key 存在，则删除
    // 这个 remove_if 方法是 c++ std::list 的方法，可以删除满足条件的元素，时
    间复杂度 O(N)
    list.remove_if([&key](KVNode& node) { return node.key == key; });
}
};

```

完整代码实现

有了上面的铺垫，我们现在来看一个比较完善代码实现，主要新增了以下几个功能：

- 1、使用了泛型，可以存储任意类型的 `key` 和 `value`。
- 2、底层的 `table` 数组会根据负载因子动态扩缩容。
- 3、使用了 [哈希表基础](#) 中提到的 `hash` 函数，利用 `key` 的 `hashCode()` 方法和 `table.length` 来计算哈希值。
- 4、实现了 `keys()` 方法，可以返回哈希表中所有的 `key`。

为了方便，我直接使用标准库的链表容器，这样就不用自己手动处理增删链表节点的操作了。当然如果你愿意的话，可以参照前文 [单/双链表代码实现](#) 自己编写操作 `KVNode` 的逻辑。

具体需要注意的细节我都写在了代码注释中，直接看代码吧：

```
#include <iostream>

#include <list>
#include <vector>

template<typename K, typename V>
class MyChainingHashMap {
    // 拉链法使用的单链表节点，存储 key-value 对
    struct KVNode {
        K key;
        V value;

        KVNode(K key, V val) : key(key), value(std::move(val)) {}
    };

    // 哈希表的底层数组，每个数组元素是一个链表，链表中每个节点是 KVNode 存储键值对
    std::vector<std::list<KVNode> > table;

    // 哈希表中存入的键值对个数
    int size_;
    // 底层数组的初始容量
    static constexpr int INIT_CAP = 4;

    // 哈希函数，将键映射到 table 的索引
    int hash(K key) {
        return (std::hash<K>{}(key) & 0x7fffffff) % table.size();
    }

    void resize(int newCap) {
        // 构造一个新的 HashMap
        // 避免 newCap 为 0，造成求模运算产生除以 0 的异常
    }
};
```



```

        newCap = std::max(newCap, 1);
        MyChainingHashMap<K, V> newMap(newCap);
        // 穷举当前 HashMap 中的所有键值对
        for (auto &list: table) {
            for (auto &node: list) {
                // 将键值对转移到新的 HashMap 中
                newMap.put(node.key, node.value);
            }
        }
        // 将当前 HashMap 的底层 table 换掉
        this->table = newMap.table;
    }
}

```

public:

```

MyChainingHashMap() : MyChainingHashMap(INIT_CAP) {
}

```

```

explicit MyChainingHashMap(int initCapacity) {

```

```

    size_ = 0;

```

情况 // 保证底层数组的容量至少为 1，因为 hash 函数中有求余运算，避免出现除以 0 的

```

    initCapacity = std::max(initCapacity, 1);

```

```

    table.resize(initCapacity);

```

```

}

```

```

// ***** 增/改 *****

```

```

// 添加 key -> val 键值对

```

```

// 如果键 key 已存在，则将值修改为 val

```

```

void put(K key, V val) {

```

```

    auto &list = table[hash(key)];

```

```

    // 如果 key 之前存在，则修改对应的 val

```

```

    for (auto &node: list) {

```

```

        if (node.key == key) {

```

```

            node.value = val;

```

```

            return;

```

```

        }

```

```

    }

```

```

    // 如果 key 之前不存在，则插入，size 增加

```

```

    list.emplace_back(key, val);

```

```

    size_++;

```

```

    // 如果元素数量超过了负载因子，进行扩容

```

```

    if (size_ >= table.size() * 0.75) {

```

```

        resize(table.size() * 2);

```

```

    }

```

```

}

```

```

// ***** 删 *****

```

```

// 删除 key 和对应的 val
void remove(K key) {
    auto &list = table[hash(key)];
    // 如果 key 存在, 则删除, size 减少
    for (auto it = list.begin(); it != list.end(); ++it) {
        if (it->key == key) {
            list.erase(it);
            size--;

            // 扩容, 当负载因子小于 0.125 时, 扩容
            if (size_ <= table.size() / 8) {
                resize(table.size() / 4);
            }
            return;
        }
    }
}

// **** 查 ****

// 返回 key 对应的 val, 如果 key 不存在, 则返回 nullptr
V get(K key) {
    const auto &list = table[hash(key)];
    for (const auto &node: list) {
        if (node.key == key) {
            return node.value;
        }
    }
    return nullptr;
}

// 返回所有 key
std::list<K> keys() {
    std::list<K> keys;
    for (const auto &list: table) {
        for (const auto &node: list) {
            keys.push_back(node.key);
        }
    }
    return keys;
}

// **** 其他工具函数 ****

int size() const {
    return size_;
}

};

int main() {

```

```

MyChainingHashMap<int, std::shared_ptr<int> > map;
map.put(1, std::make_shared<int>(1));
map.put(2, std::make_shared<int>(2));
map.put(3, std::make_shared<int>(3));
std::cout << *map.get(1) << std::endl; // 1
std::cout << *map.get(2) << std::endl; // 2

map.put(1, std::make_shared<int>(100));
std::cout << *map.get(1) << std::endl; // 100

map.remove(2);
std::cout << (map.get(2) == nullptr ? "null" :
std::to_string(*map.get(2))) << std::endl; // null

for (const auto key: map.keys()) {
    std::cout << key << " ";
}
// [1, 3] (顺序可能不同)
std::cout << std::endl;

map.remove(1);
map.remove(2);
map.remove(3);
std::cout << (map.get(1) == nullptr ? "null" :
std::to_string(*map.get(1))) << std::endl; // null

return 0;
}

```

线性探查法实现哈希表

引言

之前介绍的拉链法应该还是比较简单的，无非就是 `table` 中每个元素都是一个链表，出现哈希冲突的话往链表里塞元素就行了。

而线性探查法会更复杂，主要有两个难点，涉及到多种数组操作技巧。在讲清楚这两个难点之前，我们先设定一个简化的场景：

假设我们的哈希表只支持 `key` 类型为 `int`，`value` 类型为 `int` 的情况，且 `table.length` 固定为 10，`hash` 函数的实现是 `hash(key) = key % 10`。因为这样比较容易模拟出哈希冲突，比如 `hash(1)` 和 `hash(11)` 的值都是 1。

线性探查法的大致逻辑如下：

```

// 线性探查法的基本逻辑，伪码实现

class KVNode {
public:

```

```

    int key;
    int value;
    KVNode(int k, int v) : key(k), value(v) {}
};

class MyLinearProbingHashMap {
private:
    // 数组中每个元素都存储一个键值对
    KVNode* table[10] = {nullptr};

    int hash(int key) {
        return key % 10;
    }

public:
    // 析构函数
    ~MyLinearProbingHashMap() {
        for (int i = 0; i < 10; i++) {
            if (table[i] != nullptr) {
                delete table[i];
                table[i] = nullptr;
            }
        }
    }

    void put(int key, int value) {
        int index = hash(key);
        KVNode* node = table[index];
        if (node == nullptr) {
            table[index] = new KVNode(key, value);
        } else {
            // 线性探查法的逻辑
            // 向后探查，直到找到 key 或者找到空位
            while (index < 10 && table[index] != nullptr && table[index]-
>key != key) {
                index++;
            }
            delete table[index];
            table[index] = new KVNode(key, value);
        }
    }

    int get(int key) {
        int index = hash(key);
        // 向后探查，直到找到 key 或者找到空位
        while (index < 10 && table[index] != nullptr && table[index]->key !=
key) {
            index++;
        }
        if (index >= 10 || table[index] == nullptr) {

```

```

        return -1;
    }
    return table[index]->value;
}

void remove(int key) {
    int index = hash(key);
    // 向后探查，直到找到 key 或者找到空位
    while (index < 10 && table[index] != nullptr && table[index]->key !=
key) {
        index++;
    }
    // 删除 table[index]
    // ...
}
};

```

基于这个假设场景，我们来看看线性探查法的两个难点。

难点一：需要环形数组技巧

在线性探查的过程中，可能需要用到之前讲过的 [环形数组技巧](#)。

我们底层的 `table` 数组是一个一维数组，当发生哈希冲突时，我们需要往后找一个空位。如果一直找到数组的末尾还没找到空位，怎么办？这时候就需要转到数组的头部继续找空位，直到找到为止。

这种场景就得靠前文讲的环形数组技巧来实现。

举个具体点的例子，比方说调用一次 `put(8, a)`，底层 `table` 数组的情况是这样的：

```

table = [_, _, ..., _, a, _]
index   0 1 ... 7 8 9

```

再调用一次 `put(18, b)`，发现 `hash(18) = 8` 这个索引已经被占用了，那么就往后找一个空位：

```

table = [_, _, ..., _, a, b]
index   0 1 ... 7 8 9

```

如果再调用一次 `put(28, c)`，发现 `hash(28) = 8` 这个索引还是被占用了，那么就继续往后找空位，找到最后一个位置还没找到，就要转到数组的头部继续找：

```

table = [c, _, ..., _, a, b]
index   0 1 ... 7 8 9

```

这是 put 方法添加元素时的情况，在查找和删除元素的时候，也会遇到类似的问题，这里就不多说了。

难点二：删除操作较复杂

线性探查法的删除操作比较复杂，**因为要维护元素的连续性**。

就好比之前实现的 [动态数组](#)，因为要维护元素的连续性，所以删除和插入元素都比较麻烦。

为什么线性探查法也要维护元素的连续性呢，比方说我这样调用：

```
put(0, a)
put(10, b)
put(20, c)

put(3, A)
put(13, B)

put(30, d)
put(40, e)
```

那么底层 table 数组的情况是这样的：

```
table = [a, b, c, A, B, d, e, _, _, _]
index   0  1  2  3  4  5  6  7  8  9
hash    ^          ^
key      0 10 20  3 13 30 40
```

比方说，现在你想删除 key = 10 这个键值对，怎么办？如果你直接把 table[1] 置为 null，就会变成这样：

```
table = [a, _, c, A, B, d, e, _, _, _]
index   0  1  2  3  4  5  6  7  8  9
hash    ^          ^
key      0    20  3 13 30 40
```

这样一来元素就不连续了，回顾一下 get 方法的逻辑，此时调用 get(20), get(30), get(40) 的时候就会发现找不到对应 key，这是错误的，哈希表被搞坏了。

何解？有两种方法，我一个一个来讲。

方法一：数据搬移避免空洞

在 table 中删除元素时，可以进行类似数组的数据搬移操作，把后面的元素往前挪，保证元素的连续性。

但这里的搬移操作其实比数组的搬移操作更复杂，因为你不能把 `table[1]` 后面的全部元素往前挪，而应该仅挪动在索引 0 出现哈希冲突的元素。

举例来说，你如果移动后面的所有元素，`table` 就会变成这样：

```
table = [a, c, A, B, d, e, _, _, _, _]
index   0  1  2  3  4  5  6  7  8  9
hash    ^          ^
key      0 20   3 13  30 40
```

看到问题了么？`key = 3`，`value = A` 的那个键值对你还能找到么？不能，`hash(3) = 3`，顺着 `table[3]` 往后一直找都找不到，因为它被错误地往前移动了。

正确的做法是，只把在 `hash(key) = 0` 出现哈希冲突的元素往前挪，其他元素要不动，这样才能保证线性探查的正确性。

也就是说，删除 `key = 10` 之后，`table` 应该变成这样：

```
table = [a, c, d, A, B, e, _, _, _, _]
index   0  1  2  3  4  5  6  7  8  9
hash    ^          ^
key      0 20 30   3 13  40
```

这样一来，获取每个 `key` 对应的 `value` 就不会出错了。

但是如何能够精确地移动元素呢？这就需要一些技巧了，下篇文章会给出代码实现，这里你可以先自己思考一下。

方法二：占位符标记删除

还有一种方法，就是通过一个特殊值作为占位符来标记被删元素，这样就可以避免数据搬移，同时保证元素连续。

还是这个例子：

```
table = [a, b, c, A, B, d, e, _, _, _]
index   0  1  2  3  4  5  6  7  8  9
hash    ^          ^
key      0 10 20   3 13  30 40
```

如果你想删除 `key = 10`，那么你可以把 `table[1]` 设置为一个特殊占位符标记为删除，我用 `x` 来表示，这样 `table` 就变成了：

```
table = [a, x, c, A, B, d, e, _, _, _]
index   0  1  2  3  4  5  6  7  8  9
```


hash	^		^			
key	0	20	3	13	30	40

在 `get` 方法中会特殊处理这个特殊值，所以你调用 `get(10)` 的时候会返回 `-1`，表示没有找到。

同时，你调用 `get(20)`，`get(30)`，`get(40)` 的时候，还是可以正确地向后找到对应的 `value`。

这种方法的好处是，不需要进行数据搬移，删除操作处理起来比较简单。

但也有缺点，比如如果你不断地插入和删除元素，`table` 数组中会出现很多这样占位符，这样会增加连续元素的长度，进而降低 `get` 方法线性探查的效率。

而且还有一种很 `tricky` 的情况，比如你不断插入和删除元素，导致 `table` 中全部都是占位符。如果你不对这种情况进行特殊处理，那么此时你调用一次 `get` 方法，由于环形数组的特性，算法就会陷入死循环。

这里就提一下，具体实现可以看后面我给出的代码。

上面讲到了线性探查法的两个难点，你感觉怎么样？是不是还没看到代码，就觉得线性探查法比拉链法复杂多了？

所以线性探查法使用得不多，大部分编程语言标准库实现的哈希表都是用拉链法，因为它更简单不容易出错，而且正如我在 [哈希表原理](#) 中提到的，拉链法可以支持无限大的负载因子，而线性探查法不行。

数据结构这个东西是拿来用的，而不是拿来秀的，并不是越复杂越好，简单实用、易学易维护才是王道。

线性探查法的两种代码实现

简化版代码

简化实现中，具体简化的地方如下：

- 1、我们实现的哈希表只支持 `key` 类型为 `int`，`value` 类型为 `int` 的情况，如果 `key` 不存在，就返回 `-1`。
- 2、我们实现的 `hash` 函数就是简单地取模，即 `hash(key) = key % table.length`。这样也方便模拟出哈希冲突的情况，比如当 `table.length = 10` 时，`hash(1)` 和 `hash(11)` 的值都是 1。
- 3、底层的 `table` 数组的大小在创建哈希表时就固定，假设 `table` 数组不会被装满，不考虑负载因子和动态扩缩容的问题。

搬移数据的线性探查法

这种方法在 `remove` 操作时，会将删除的元素后面的元素重新插入哈希表，以此保持元素的连续性。

```
#include <iostream>
#include <vector>

using namespace std;

// 用线性探查法解决哈希冲突的简化实现（rehash 版）
class Node {
public:
    int key;
    int val;

    Node(int key, int val) {
        this->key = key;
        this->val = val;
    }
};

class ExampleLinearProbingHashMap1 {
private:
    // 哈希表的底层数组，每个索引存储一个键值对
    vector<Node*> table;

public:
    ExampleLinearProbingHashMap1(int cap) {
        this->table.resize(cap);
    }

    ~ExampleLinearProbingHashMap1() {
        for (Node* node : table) {
            if (node != NULL) {
                delete node;
            }
        }
    }

    // 增/改
    void put(int key, int value) {
        int index = findKeyIndex(key);
        table[index] = new Node(key, value);
    }

    // 查，找不到就返回 -1
    int get(int key) {
        int index = findKeyIndex(key);
        return table[index] == NULL ? -1 : table[index]->val;
    }
};
```

```

// 删
void remove(int key) {
    int index = findKeyIndex(key);
    if (table[index] == NULL) {
        return;
    }
    table[index] = NULL;
    // 保持元素连续性, 搬移数据 (这个过程称为 rehash)
    index = (index + 1) % table.size();
    while (table[index] != NULL) {
        Node* entry = table[index];
        table[index] = NULL;
        // 这个操作是关键, 利用 put 方法, 将键值对重新插入
        // 这样就能把它们移动到正确的 table 索引位置
        put(entry->key, entry->val);
        index = (index + 1) % table.size();
    }
}

// 线性探测法查找 key 在 table 中的索引
// 如果找不到, 返回的就是下一个为 null 的索引, 可用于插入
int findKeyIndex(int key) {
    int index = hash(key);

    while (table[index] != NULL) {
        if (table[index]->key == key) {
            return index;
        }
        // 注意环形数组特性
        index = (index + 1) % table.size();
    }

    return index;
}

int hash(int key) {
    return key % table.size();
}

};

int main() {
    ExampleLinearProbingHashMap1 map(10);
    map.put(1, 1);
    map.put(2, 2);
    map.put(10, 10);
    map.put(20, 20);
    map.put(30, 30);
    map.put(3, 3);
    cout << map.get(1) << endl;
}

```

```

    cout << map.get(2) << endl;
    cout << map.get(20) << endl;

    map.put(1, 100);
    cout << map.get(1) << endl;

    map.remove(20);
    cout << map.get(20) << endl;
    cout << map.get(30) << endl;

    return 0;
}

```

特殊占位符的线性探查法

这种方法通过一个 `DELETED` 特殊值作为占位符，标记被删除的元素。

这个方法与上面的方法最大的区别在于 `findKeyIndex` 方法的实现，同时需要对 `DELETED` 特殊处理。直接看代码吧，后面会通过可视化面板展示增删查改的过程：

```

#include <iostream>
#include <vector>

// 用线性探查法解决哈希冲突的简化实现（特殊占位符版）
// Special placeholder for deleted elements.
// 用于标记被删元素的占位符
struct KVNode {
    int key;
    int val;
};

class ExampleLinearProbingHashMap2 {
private:
    // 真正存储键值对的数组
    std::vector<KVNode*> table;
    // 里面的值可以随意存储，因为只会使用 == 判断指针相等，不用比较里面的值
    KVNode* DELETED = new KVNode{-2, -2};

    // 哈希函数，将键映射到 table 的索引
    int hash(int key) {
        return key % table.size();
    }

    // 线性探测法查找 key 在 table 中的索引，如果找不到，返回 -1
    int findKeyIndex(int key) {
        // 因为删除元素时只是标记为 DELETED，并不是真的删除，所以 table 可能会被填满，导致死循环
        // step 用来记录查找的步数，防止死循环
        for (int i = hash(key), step = 0; table[i] != nullptr; i = (i+1) %

```

```

table.size()) {
    if (++step > table.size()) return -1;
    // 遇到占位符直接跳过
    if (table[i] == DELETED) continue;
    if (table[i]->key == key) return i;
}

return -1;
}

public:
    // 构造函数，初始化哈希表容量
    explicit ExampleLinearProbingHashMap2(int initCapacity) {
        table.resize(initCapacity, nullptr);
    }

    // 增/改
    void put(int key, int val) {
        int index = findKeyIndex(key);
        // key 已存在，修改对应的 val，如果 key 不存在，新建节点并插入表中
        if (index != -1 && table[index] != nullptr) {
            table[index]->val = val;
            return;
        }

        KVNode* node = new KVNode{key, val};
        index = hash(key);
        while (table[index] != nullptr && table[index] != DELETED) {
            index = (index+1) % table.size();
        }
        table[index] = node;
    }

    // 删
    void remove(int key) {
        int index = findKeyIndex(key);
        // key 不存在，不需要 remove
        if (index == -1) return;
        // 直接用占位符表示删除
        table[index] = DELETED;
    }

    // 查，返回 key 对应的 val，如果 key 不存在，则返回 -1
    int get(int key) {
        int index = findKeyIndex(key);
        return (index != -1) ? table[index]->val : -1;
    }
};

int main() {

```

```

ExampleLinearProbingHashMap2 map(10);
map.put(1, 1);
map.put(2, 2);
map.put(10, 10);
map.put(20, 20);
map.put(30, 30);
map.put(3, 3);
std::cout << map.get(1) << std::endl; // Output: 1
std::cout << map.get(2) << std::endl; // Output: 2
std::cout << map.get(20) << std::endl; // Output: 20

map.put(1, 100);
std::cout << map.get(1) << std::endl; // Output: 100

map.remove(20);
std::cout << map.get(20) << std::endl; // Output: -1
std::cout << map.get(30) << std::endl; // Output: 30

return 0;
}

```

完整版代码

有了上面的铺垫，我们现在来看比较完善的 Java 代码实现，主要新增了以下几个功能：

- 1、使用了泛型，可以存储任意类型的 key 和 value。
- 2、底层的 table 数组会根据负载因子动态扩缩容。
- 3、使用了 [哈希表基础](#) 中提到的 hash 函数，利用 key 的 hashCode() 方法和 table.length 来计算哈希值。
- 4、实现了 keys() 方法，可以返回哈希表中所有的 key。

下面我会分别给出 rehash 版本和特殊值标记版本的实现，具体细节可以参考代码注释。

rehash 版本

```

#include <iostream>
#include <vector>
#include <list>
#include <stdexcept>

template<typename K, typename V>
class MyLinearProbingHashMap1 {
private:
    struct KVNode {
        K key;
        V val;
    };

```

```

        KVNode(K key, V val) : key(key), val(val) {
        }
};

std::vector<KVNode *> table;
int size_;
static const int INIT_CAP = 4;

public:
    MyLinearProbingHashMap1() : MyLinearProbingHashMap1(INIT_CAP) {
    }

    MyLinearProbingHashMap1(int initCapacity) : size_(0) {
        table.resize(initCapacity, nullptr);
    }

    // **** 增/改 ****
    void put(K key, V val) {
        // 我们把负载因子默认设为 0.75, 超过则扩容
        if (size_ >= table.size() * 0.75) {
            resize(table.size() * 2);
        }

        int index = getKeyIndex(key);
        // key 已存在, 修改对应的 val
        if (table[index] != nullptr) {
            table[index]->val = val;
            return;
        }

        // key 不存在, 在空位插入
        table[index] = new KVNode(key, val);
        size_++;
    }

    // **** 删 ****
    // 删除 key 和对应的 val
    void remove(K key) {
        // 缩容, 当负载因子小于 0.125 时, 缩容
        if (size_ <= table.size() / 8) {
            resize(table.size() / 4);
        }

        int index = getKeyIndex(key);
        if (table[index] == nullptr) {
            // key 不存在, 不需要 remove
            return;
        }

        // 开始 remove

```

```

        delete table[index];
        table[index] = nullptr;
        size--;
        // 保持元素连续性, 进行 rehash
        index = (index + 1) % table.size();
        while (table[index] != nullptr) {
            KVNode *entry = table[index];
            table[index] = nullptr;
            size--;
            put(entry->key, entry->val);
            delete entry;
            index = (index + 1) % table.size();
        }
    }
}

```

// **** 查 ****

// 返回 key 对应的 val, 如果 key 不存在, 则返回 null

```

V get(K key) {
    int index = getKeyIndex(key);
    if (table[index] == nullptr) {
        return V(); // 返回默认值
    }
    return table[index]->val;
}

```

// 返回所有 key (顺序不固定)

```

std::list<K> keys() {
    std::list<K> keys;
    for (KVNode *entry: table) {
        if (entry != nullptr) {
            keys.push_back(entry->key);
        }
    }
    return keys;
}

```

// **** 其他工具函数 ****

```

int size() const {
    return size_;
}

```

private:

// 哈希函数, 将键映射到 table 的索引

```

int hash(K key) {
    return (std::hash<K>()(key) & 0x7fffffff) % table.size();
}

```

// 对 key 进行线性探查, 返回一个索引

```

int getKeyIndex(K key) {
    int index;

```



```

        for (index = hash(key); table[index] != nullptr; index = (index + 1)
% table.size()) {
            if (table[index]->key == key) {
                return index;
            }
        }
        return index;
    }

    void resize(int newCap) {
        MyLinearProbingHashMap1<K, V> newMap(newCap);
        for (KVNode *entry: table) {
            if (entry != nullptr) {
                newMap.put(entry->key, entry->val);
            }
        }
        table = std::move(newMap.table);
    }
};

int main() {
    MyLinearProbingHashMap1<int, int> map;
    map.put(1, 1);
    map.put(2, 2);
    map.put(10, 10);
    map.put(20, 20);
    map.put(30, 30);
    map.put(3, 3);
    std::cout << map.get(1) << std::endl; // 1
    std::cout << map.get(2) << std::endl; // 2
    std::cout << map.get(20) << std::endl; // 20

    map.put(1, 100);
    std::cout << map.get(1) << std::endl; // 100

    map.remove(20);
    std::cout << map.get(20) << std::endl; // 0 (null)
    std::cout << map.get(30) << std::endl; // 30

    return 0;
}

```

特殊值标记版本

```

#include <iostream>
#include <vector>
#include <list>
#include <stdexcept>

```

```

template<typename K, typename V>
class MyLinearProbingHashMap2 {
    // 键值对节点
    struct KVNode {
        K key;
        V val;

        KVNode(K key, V val) : key(key), val(val) {
        }
    };

    // 被删除的 KVNode 的占位符
    KVNode *DUMMY = new KVNode(K(), V());

    // 真正存储键值对的 table 数组
    std::vector<KVNode *> table;
    // HashMap 中的键值对个数
    int size;
    // 默认的初始化容量
    static const int INIT_CAP = 4;

public:
    MyLinearProbingHashMap2() : MyLinearProbingHashMap2(INIT_CAP) {
    }

    MyLinearProbingHashMap2(int cap) : size(0), table(cap, nullptr) {
    }

    // **** 增/改 ****

    // 添加 key -> val 键值对
    // 如果键 key 已存在，则将值修改为 val
    void put(K key, V val) {
        if (key == K()) {
            throw std::invalid_argument("key is null");
        }

        // 负载因子默认设为 0.75，超过则扩容
        if (size >= table.size() * 0.75) {
            resize(table.size() * 2);
        }

        int index = getKeyIndex(key);
        if (index != -1) {
            // key 已存在，修改对应的 val
            table[index]->val = val;
            return;
        }

        // key 不存在

```

```

    KVNode *x = new KVNode(key, val);
    // 在 table 中找一个空位或者占位符, 插入
    index = hash(key);
    while (table[index] != nullptr && table[index] != DUMMY) {
        index = (index + 1) % table.size();
    }
    table[index] = x;
    size++;
}

// **** 删 ****

// 删除 key 和对应的 val, 并返回 val
// 若 key 不存在, 则返回 null
void remove(K key) {
    if (key == K()) {
        throw std::invalid_argument("key is null");
    }

    // 扩容
    if (size < table.size() / 8) {
        resize(table.size() / 2);
    }

    int index = getKeyIndex(key);
    if (index == -1) {
        // key 不存在, 不需要 remove
        return;
    }

    // 开始 remove
    // 直接用占位符表示删除
    table[index] = DUMMY;
    size--;
}

// **** 查 ****

// 返回 key 对应的 val
// 如果 key 不存在, 则返回 null
V get(K key) {
    if (key == K()) {
        throw std::invalid_argument("key is null");
    }

    int index = getKeyIndex(key);
    if (index == -1) {
        return V();
    }
}

```

```

        return table[index]->val;
    }

    bool containsKey(K key) {
        return getKeyIndex(key) != -1;
    }

    std::list<K> keys() {
        std::list<K> keys;
        for (auto entry: table) {
            if (entry != nullptr && entry != DUMMY) {
                keys.push_back(entry->key);
            }
        }
        return keys;
    }

    int getSize() {
        return size;
    }

private:
    // 对 key 进行线性探查，返回一个索引
    // 根据 keys[i] 是否为 null 判断是否找到对应的 key
    int getKeyIndex(K key) {
        int step = 0;
        for (int i = hash(key); table[i] != nullptr; i = (i + 1) %
table.size()) {
            step++;
            // 防止死循环
            if (step > table.size()) {
                // 这里可以触发一次 resize，把标记为删除的占位符清理掉
                resize(table.size());
                return -1;
            }
            KNode *entry = table[i];
            // 遇到占位符直接跳过
            if (entry == DUMMY) {
                continue;
            }
            if (entry->key == key) {
                return i;
            }
        }
        return -1;
    }

    // 哈希函数，将键映射到 table 的索引
    // [0, table.length - 1]
    int hash(K key) {

```

```

        return (std::hash<K>{}(key) & 0x7fffffff) % table.size();
    }

    void resize(int cap) {
        MyLinearProbingHashMap2<K, V> newMap(cap);
        for (auto entry: table) {
            if (entry != nullptr && entry != DUMMY) {
                newMap.put(entry->key, entry->val);
            }
        }
        this->table = newMap.table;
    }
};

int main() {
    MyLinearProbingHashMap2<int, int> map;
    map.put(1, 1);
    map.put(2, 2);
    map.put(10, 10);
    map.put(20, 20);
    map.put(30, 30);
    map.put(3, 3);
    std::cout << map.get(1) << std::endl; // 1
    std::cout << map.get(2) << std::endl; // 2
    std::cout << map.get(20) << std::endl; // 20

    map.put(1, 100);
    std::cout << map.get(1) << std::endl; // 100

    map.remove(20);

    std::cout << map.containsKey(20) << std::endl; // 0 (false)
    std::cout << map.get(20) << std::endl; // 0 (null)
    std::cout << map.get(30) << std::endl; // 30
}

```

哈希集合的原理及代码实现

哈希集合原理

哈希集合的主要使用场景是「去重」，因为它的特性是：**不会出现重复元素**，可以在 $O(1)$ 的时间增删元素，可以在 $O(1)$ 的时间判断一个元素是否存在。

哈希集合的主要 API 如下：

```

class HashSet {
public:
    // 增，向哈希集合中添加一个元素，复杂度  $O(1)$ 
    // 如果元素已存在，则什么都不会发生

```

```
void add(int key);

// 删，从哈希集合中移除一个元素，复杂度 O(1)
// 如果元素不存在，则什么都不会发生
void remove(int key);

// 查，判断元素是否存在，复杂度 O(1)
bool contains(int key);
};
```

这几个 API 咋实现？其实根本用不着实现，直接把前文教给你的哈希表拿来用就行了。

你往哈希表里插入一个键值对 `put(key, value)`，时间复杂度是 $O(1)$ ；你往哈希表里删除一个键值对 `remove(key)`，时间复杂度是 $O(1)$ ；判断一个键是否存在哈希表中，就是判断 `get(key)` 是否得到空指针 `null`，所以时间复杂度也是 $O(1)$ 。

综上，这几个哈希集合的 API 可以直接复用哈希表的 API 来实现。操作哈希集合，其实就是操作哈希表的键。

那就有读者问了，哈希表里面的值呢，如何处理？答案是不处理，直接忽略掉就行了，我们可以用一个占位符来填充值，等会看代码实现你就懂了。

虽然只是简单封装了哈希表，但是大部分高级编程语言还是提供哈希集合的数据结构的，比如 Python 的 `set`，Java 的 `HashSet`，C++ 的 `unordered_set` 等等。但是像 Go 语言，标准库干脆就不提供哈希集合这种数据结构，你要用的话需要自己用哈希表 `map` 来模拟。

因为哈希集合的元素就是哈希表里面的键，所以**哈希集合和哈希表具有相同的限制**，比如不能依赖哈希集合的元素遍历顺序、哈希集合中的元素应该是不可变的等等，具体限制和原因可以回顾前文 [哈希表特性及原理](#)。

哈希集合代码实现

有了上面的铺垫，代码实现应该没有任何难度了。

```
#include <unordered_map>

template<typename K>
class MyHashSet {
private:
    // 底层 unordered_map，用于存储集合元素
    std::unordered_map<K, bool> map;

public:
    void add(const K& key) {
        // 向哈希表添加一个键值对，值用 true 作为占位符
        map[key] = true;
    }
};
```

```

void remove(const K& key) {
    // 从哈希表中移除键 key
    map.erase(key);
}

bool contains(const K& key) {
    // 判断哈希表中是否包含键 key
    return map.find(key) != map.end();
}

int size() {
    return map.size();
}
};

```

哈希表结构的变换

用链表加强哈希表（LinkedHashMap）

引言

前文从原理上分析了，不能依赖哈希表遍历 `key` 的顺序，即哈希表中的 `key` 是无序的。

但结合实际的编程经验，你可能会有些疑问。

比如熟悉 Python 的读者可能知道，Python 3.7 开始，标准库提供的哈希表 `dict` 就明确告诉你了，`dict` 的键的遍历顺序就是键的插入顺序。比如下面这段简单的代码：

```

d = dict()

d['a'] = 1
d['b'] = 2
d['c'] = 3
print(list(d.keys())) # ['a', 'b', 'c']

d['y'] = 4
print(list(d.keys())) # ['a', 'b', 'c', 'y']

d['d'] = 5
print(list(d.keys())) # ['a', 'b', 'c', 'y', 'd']

```

无论你插入多少键，`keys` 方法返回的所有键都是按照插入顺序排列，感觉就好像在向数组尾部追加元素一样。这怎么可能呢？

如果你熟悉 Golang，你会发现一个更神奇的现象。比如下面这段测试代码：

```

package main

```

```

import (
    "fmt"
)

func main() {
    // 初始化 map
    myMap := map[string]int{
        "1": 1,
        "2": 2,
        "3": 3,
        "4": 4,
        "5": 5,
    }

    // 定义遍历 map 的函数
    printMapKeys := func(m map[string]int) {
        for key := range m {
            fmt.Print(key, " ")
        }
        fmt.Println()
    }

    // 多次遍历 map，观察键的顺序
    printMapKeys(myMap)
    printMapKeys(myMap)
    printMapKeys(myMap)
    printMapKeys(myMap)
}

// 我运行的结果如下：
// 1 2 3 4 5
// 5 1 2 3 4
// 2 3 4 5 1
// 1 2 3 4 5

```

也就是说，它每次遍历的顺序都是随机。但是按照前文 [哈希表原理](#) 所说，虽然哈希表的键是无序的，但是没有对哈希表做任何操作，遍历得到的结果应该不会变才对，Golang 的 map 每次遍历的顺序咋都不一样？这也太离谱了吧？

你可以先自己思考下原因，下面我给出答案。

先说 Golang 吧，每次遍历都乱序的原因就是，它故意的。

这个原因属实是让人有些哭笑不得，Golang 为了防止开发者依赖哈希表的遍历顺序，所以每次遍历都故意返回不同的顺序，可谓用心良苦。也可以从侧面看出，确实不少开发者没了解过哈希表的基本原理。

我们不妨进一步想想，它是怎么打乱顺序的呢？真是随机打乱吗？

其实不是，你仔细看看，它这个乱序是有规律的。有没有想起前面讲过的 [环形数组](#)？

```
| 1 2 3 4 5
5 | 1 2 3 4
2 3 4 5 | 1
| 1 2 3 4 5
```

看出来没有？如果不触发扩缩容的话，实际上它的遍历顺序应该也是固定的，只不过它不是每次都从底层 `table` 数组的头部开始，而是从一个随机的位置开始，然后利用环形数组技巧遍历整个 `table` 数组，这样就能保证多次遍历的结果具有随机性，同时又不至于为了随机性而牺牲太多性能。

再说 Python，它能让所有键按照插入顺序排列，是因为它把标准的哈希表和链表结合起来，组成了一种新的数据结构：哈希链表。

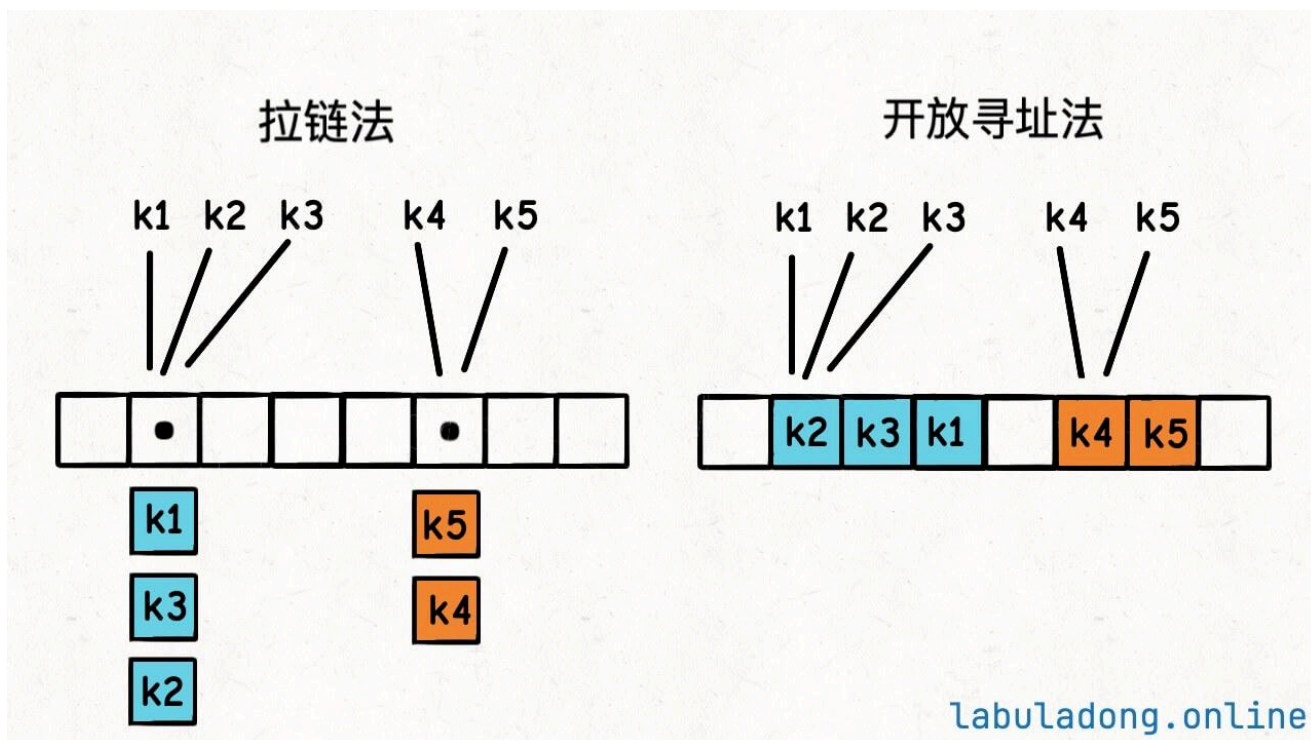
其他编程语言也有类似的实现，比如 Java 的 `LinkedHashMap`。这种数据结构兼具了哈希表 $O(1)$ 的增删查改效率，同时又可以像数组链表一样保持键的插入顺序。

它是怎么做到的呢？下面我会具体讲解。

哈希链表（LinkedHashMap）实现思路

我们先明确一下问题。

标准哈希表的键是无序存储在底层的 `table` 数组中的：

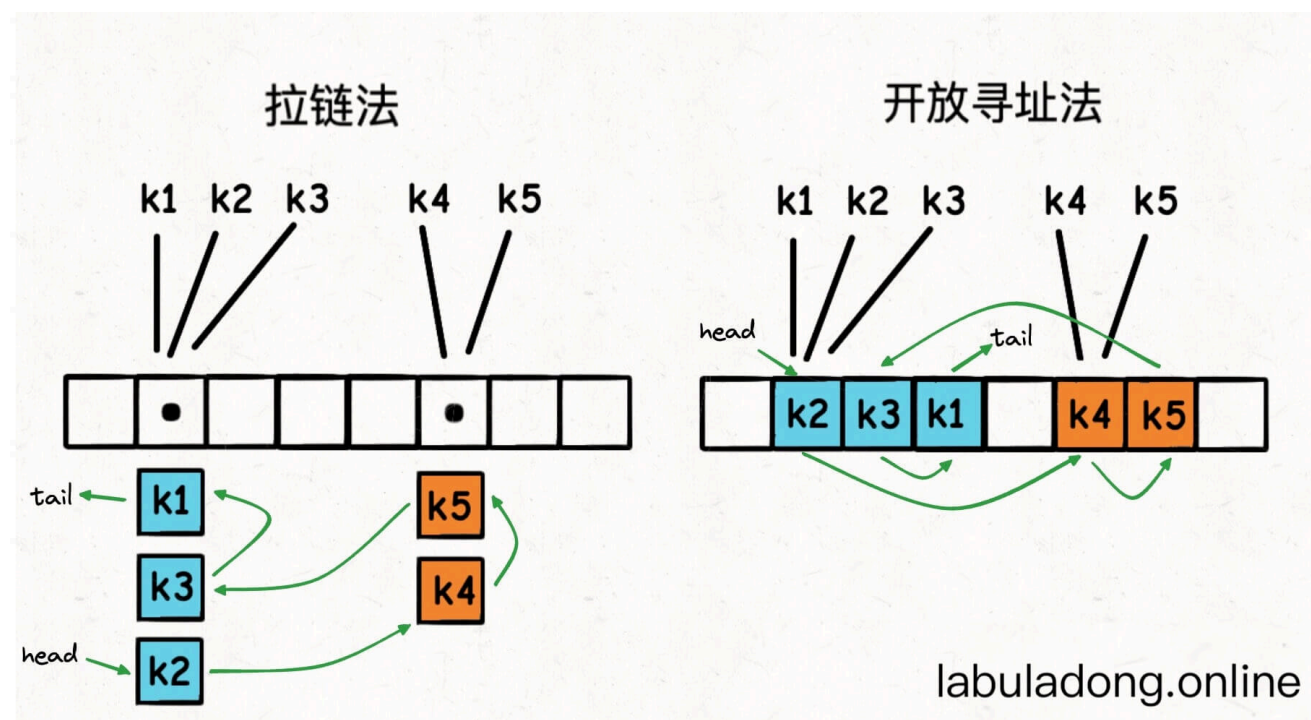


你光看这个图，看不出来这些键是按什么顺序插入的，且一旦触发扩缩容，这键的位置又会改变。

我们现在希望在不改变哈希表增删查改复杂度的前提下，能够按照插入顺序来访问所有键，且不受扩缩容影响。

那么一个最直接的思路就是，我想办法把这些键值对都用类似链表节点的结构串联起来，持有一个头尾结点 `head`, `tail` 的引用，每次把新的键插入 `table` 数组时，同时把这个键插入链表的尾部。

这样一来，只要我从头结点 `head` 开始遍历链表，就能按照插入顺序访问所有键了：



我们可以清楚地看出，键的插入顺序是 `k2`, `k4`, `k5`, `k3`, `k1`。

发挥抽象思维，不要陷入细节

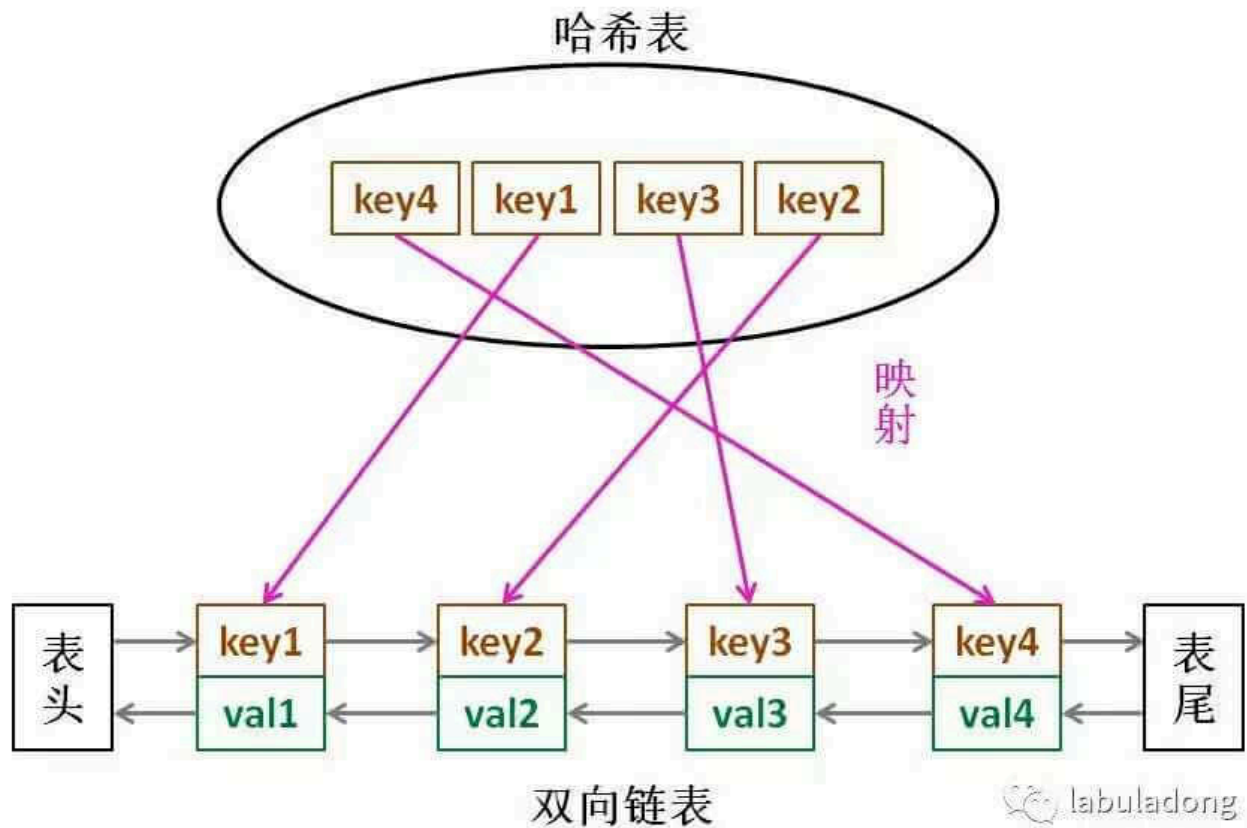
但肯定有读者会绕晕：之前 [拉链法实现哈希表](#) 不就是用链表解决哈希冲突的吗？现在相当于在链表上再套一层链表？这咋实现啊？

所以我在本站的开篇就说，要有抽象能力。你看所有数据结构，本质上就是在数组、链表这两种基本结构上雕花，但即便如此，多套几层你也晕了。

如果产生这样的疑问，是因为你脑子里全是前两章讲的拉链法、线性探查法的底层实现原理，陷入细节无法自拔了，是不是这样？

请跳出细节，抽象来看，哈希表本质上就是一个键值映射，链表本质上就是一个顺序存储元素的容器。现在我就是想让这个键值映射中的键按照插入顺序排列，怎么把哈希表和链表结合起来？

答案是这样：



假设键和值都是字符串类型，标准的哈希表是这样：

```
HashMap<String, String> map = new HashMap<>();

// 插入键值对
String key = "k1";
String value = "v1";

map.put(key, value);
```

而我们现在给哈希表的值类型套了一层双链表结构：

```
// 双链表节点
class Node {
    String key;
    String value;
    Node prev;
    Node next;

    Node(String key, String value) {
        this.key = key;
        this.value = value;
    }
}

HashMap<String, Node> map = new HashMap<>();
```

```
// 插入键值对
String key = "k1";
String value = "v1";

map.put(key, new Node(key, value));

// 这里做了简化，实际实现中还要操作新的链表节点加入链表
```

这样一来，就实现了哈希链表结构：

- 我们还是可以在 $O(1)$ 的时间复杂度内通过键查找到对应的双链表节点，进而找到键对应的值。
- 我们可以在 $O(1)$ 的时间复杂度内插入新的键值对。因为哈希表本身的插入操作时间复杂度是 $O(1)$ ，且双链表头尾的插入操作时间复杂度也是 $O(1)$ 。
- 我们可以在 $O(1)$ 的时间复杂度内删除指定的键值对。因为哈希表本身的删除操作时间复杂度是 $O(1)$ ，**删除给定双链表节点的操作时间复杂度也是 $O(1)$** 。
- 由于链表节点的顺序是插入顺序，那么只要从头结点开始遍历这个链表，就能按照插入顺序访问所有键。

也就是说，我们在不改变标准哈希表的基本操作复杂度的前提下，实现了按照插入顺序访问所有键的需求。

链表的删除操作时间复杂度是 $O(1)$ ？

肯定有读者记得，前文 [链表原理](#) 说过，链表的删除操作时间复杂度是 $O(n)$ 啊，怎么现在又说是 $O(1)$ 了？

实际上我说的已经很严谨了，我这里说的是 **删除给定双链表节点的操作是 $O(1)$** 。

前文我们动手实现链表的时候，我们是删除指定索引的链表节点，时间复杂度确实是 $O(n)$ ，因为要先遍历到这个索引，然后才能开始删除嘛。

但是这里并没有遍历的过程，我们是直接通过哈希表键值映射拿到的这个链表节点，时间复杂度是 $O(1)$ 。

且双链表节点只需进行简单的指针操作即可把自身从链表中删除，时间复杂度是 $O(1)$ ：

```
// 删除给定双链表节点
Node target;

// 在双链表中删除 target 节点
target.prev.next = target.next;
target.next.prev = target.prev;
target.prev = null;
target.next = null;
```

所以哈希链表删除操作的复杂度依然是 $O(1)$ 。

这里需要注意，双链表节点同时拥有前后驱指针，才可以做到 $O(1)$ 时间复杂度的删除操作。单链表节点只有后驱指针，但没有前驱指针，做不到 $O(1)$ 时间复杂度的删除操作。**所以哈希链表的实现中，只能使用双链表。**

代码实现

明白了哈希链表的原理，直接看代码实现吧，比较简单：

```
#include <iostream>
#include <unordered_map>
#include <vector>

using namespace std;

template <typename K, typename V>
struct Node {
    K key;
    V val;
    Node* next;
    Node* prev;

    Node(K key, V val) : key(key), val(val), next(nullptr), prev(nullptr) {}
};

template <typename K, typename V>
class MyLinkedHashMap {
public:
    MyLinkedHashMap() {
        head = new Node<K, V>(K(), V());
        tail = new Node<K, V>(K(), V());
        head->next = tail;
        tail->prev = head;
    }

    // 析构函数，释放所有节点内存
    ~MyLinkedHashMap() {
        // 释放所有数据节点
        Node<K, V>* current = head->next;
        while (current != tail) {
            Node<K, V>* next = current->next;
            delete current;
            current = next;
        }
        // 释放头尾节点
        delete head;
    }
};
```

```

        delete tail;
    }

    // 禁用拷贝构造和赋值操作，避免浅拷贝问题
    MyLinkedHashMap(const MyLinkedHashMap&) = delete;
    MyLinkedHashMap& operator=(const MyLinkedHashMap&) = delete;

    V get(K key) {
        if (map.find(key) == map.end()) {
            return V();
        }
        return map[key]->val;
    }

    void put(K key, V val) {
        // 若为新插入的节点，则同时插入链表和 map
        if (map.find(key) == map.end()) {
            // 插入新的 Node
            Node<K, V>* node = new Node<K, V>(key, val);
            addLastNode(node);
            map[key] = node;
            return;
        }
        // 若存在，则替换之前的 val
        map[key]->val = val;
    }

    void remove(K key) {
        // 若 key 本不存在，直接返回
        if (map.find(key) == map.end()) {
            return;
        }
        // 若 key 存在，则需要同时在哈希表和链表中删除
        Node<K, V>* node = map[key];
        map.erase(key);
        removeNode(node);
        // 释放节点内存
        delete node;
    }

    bool containsKey(K key) {
        return map.find(key) != map.end();
    }

    vector<K> keys() {
        vector<K> keyList;
        for (Node<K, V>* p = head->next; p != tail; p = p->next) {
            keyList.push_back(p->key);
        }
        return keyList;
    }

```

```

    }

    // 获取元素数量
    size_t size() {
        return map.size();
    }

    // 判断是否为空
    bool empty() {
        return map.empty();
    }

private:

    Node<K, V>* head;
    Node<K, V>* tail;
    unordered_map<K, Node<K, V>*> map;

    void addLastNode(Node<K, V>* x) {
        Node<K, V>* temp = tail->prev;
        // temp <-> tail
        x->next = tail;
        x->prev = temp;

        temp->next = x;
        tail->prev = x;
    }

    void removeNode(Node<K, V>* x) {
        Node<K, V>* prev = x->prev;
        Node<K, V>* next = x->next;
        // prev <-> x <-> next

        prev->next = next;
        next->prev = prev;

        x->next = x->prev = nullptr;
    }
};

int main() {
    MyLinkedHashMap<string, int> map;
    map.put("a", 1);
    map.put("b", 2);
    map.put("c", 3);
    map.put("d", 4);
    map.put("e", 5);

    // output: a b c d e

```

```

    for (const auto& key : map.keys()) {
        cout << key << " ";
    }
    cout << endl;

    map.remove("c");
    cout << map.containsKey("c") << endl;

    // output: a b d e
    for (const auto& key : map.keys()) {
        cout << key << " ";
    }
    cout << endl;

    return 0;
}

```

LinkedHashSet 的实现

LinkedHashSet 就是一个可以保持插入顺序的哈希集合。

前文 [哈希集合的原理及实现](#) 说过，哈希集合的代码实现其实就是包装了一下哈希表。

所以这里 LinkedHashSet 的实现也是直接包装一下 MyLinkedHashMap 即可，非常简单，我就不写代码了。

用数组加强哈希表（ArrayHashMap）

添加 randomKey() API

现在我给你出个题，让你基于标准哈希表的 API 之上，再添加一个新的 randomKey() API，可以在 O(1) 的时间复杂度返回一个随机键：

```

interface Map<K, V> {
    // 获取 key 对应的 value，时间复杂度 O(1)
    V get(K key);

    // 添加/修改 key-value 对，时间复杂度 O(1)
    void put(K key, V value);

    // 删除 key-value 对，时间复杂度 O(1)
    void remove(K key);

    // 是否包含 key，时间复杂度 O(1)
    boolean containsKey(K key);

    // 返回所有 key，时间复杂度 O(N)
    List<K> keys();
}

```



```
// 新增 API: 随机返回一个 key, 要求时间复杂度 O(1)
K randomKey();
}
```

均匀随机 (uniform random)

注意, 我们一般说的随机, 都是指均匀随机, 即每个元素被选中的概率相等。比如你有 n 个元素, 你的随机算法要保证每个元素被选中的概率都是 $1/n$, 才叫均匀随机。

怎么样, 你会不会做? 不要小看这个简单的需求, 实现方法其实是比较巧妙的。

通过前面的学习, 你应该知道哈希表的本质就是一个 `table` 数组, 现在让你随机返回一个哈希表的键, 很容易就会联想到在数组中随机获取一个元素。

在标准数组, 随机获取一个元素很简单, 只要用随机数生成器生成一个 $[0, \text{size})$ 的随机索引, 就相当于找了一个随机元素:

```
int randomeElement(vector<int>& arr) {
    // 生成 [0, arr.size()) 的随机索引
    return arr[rand() % arr.size()];
}
```

这个算法是正确的, 它的复杂度是 $O(1)$, 且每个元素被选中的概率都是 $1/n$, n 为 `arr` 数组的总元素个数。

但你注意, 上面这个函数有个前提, 就是数组中的元素是紧凑存储没有空洞的, 比如 `arr = [1, 2, 3, 4]`, 这样才能保证任意一个随机索引都对应一个有效的元素。

如果数组中有空洞就有问题了, 比如 `arr = [1, 2, null, 4]`, 其中 `arr[2] = null` 代表没有存储元素的空洞, 那么如果你生成的随机数恰好是 2, 请问你该怎么办?

也许你想说, 可以向左或者向右线性查找, 找到一个非空的元素返回, 类似这样:

```
// 返回一个非空的随机元素 (伪码)
int randomeElement(int[] arr) {
    Random r = new Random();
    // 生成 [0, arr.length) 的随机索引
    int i = r.nextInt(arr.length);
    while (arr[i] == null) {
        // 随机生成的索引 i 恰巧是空洞
        // 借助环形数组技巧向右进行探查
        // 直到找到一个非空元素
        i = (i + 1) % arr.length;
    }
    return arr[i];
}
```

你这样是不行的, 这个算法有两个问题:

- 1、有个循环，最坏时间复杂度上升到了 $O(N)$ ，不符合 $O(1)$ 的要求。
- 2、这个算法不是均匀随机的，因为你的查找方向是固定的，空洞右侧的元素被选中的概率会更大。比如 `arr = [1, 2, null, 4]`，元素 1, 2, 4 被选中的概率分别是 $1/4$, $1/4$, $2/4$ 。

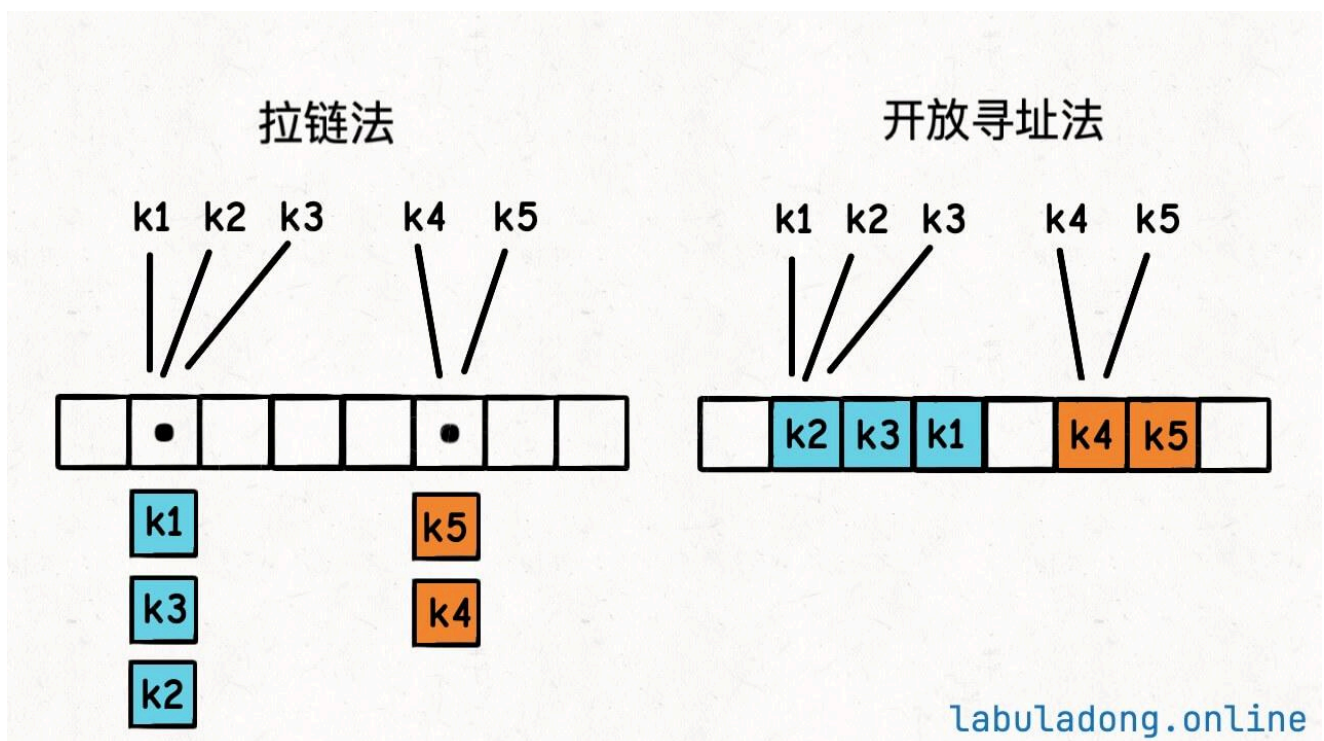
那也许还有个办法，一次运气不好，就多来随机几次，直到找到一个非空元素：

```
// 返回一个非空的随机元素（伪码）
int randomeElement(int[] arr) {
    Random r = new Random();
    // 生成 [0, arr.length) 的随机索引
    int i = r.nextInt(arr.length);
    while (arr[i] == null) {
        // 随机生成的索引 i 恰巧是空洞
        // 重新生成一个随机索引
        i = r.nextInt(arr.length);
    }
    return arr[i];
}
```

现在这个算法是均匀随机的，但问题也非常明显，它的时间复杂度竟然依赖随机数！肯定不是 $O(1)$ 的，不符合要求。

怎么样，从一个带有空洞的数组中随机返回一个元素是不是都把你难住了？

别忘了，我们现在的目标是从哈希表中随机返回一个键，**哈希表底层的 table 数组不仅包含空洞，情况还会更复杂一些：**



如果你的哈希表用开放寻址法解决哈希冲突，那还好，就是带空洞数组的场景。

如果你的哈希表用拉链法，那可麻烦了。数组里面的每个元素是一个链表，你光随机一个索引是不够的，还要随机链表中的一个节点。

而且注意概率，这个拉链法，就算你均匀随机到一个数组索引，又均匀随机该索引存储的链表节点，得到的这个键是均匀随机的么？

其实不是，上图中 k_1, k_2, k_3 被随机到的概率是 $1/2 * 1/3 = 1/6$ ，而 k_4, k_5 被随机到的概率是 $1/2 * 1/2 = 1/4$ ，这不是均匀随机。

关于概率算法

概率算法也是非常有意思的一类问题，无论算法题还是实际业务中都会用到一些经典的随机算法，我会在后文 [谈谈游戏中的随机算法](#) 和 [带权重的随机选择](#) 中详细讲解，这里暂时不需要掌握。

唯一的办法就是通过 `keys` 方法遍历整个 `table` 数组，把所有的键都存储到一个数组中，然后再随机返回一个键。但这样复杂度就是 $O(N)$ 了，还是不符合要求。

是不是感觉已经走投无路了？所以说，还是要积累一些经典数据结构设计经验，如果面试笔试的时候遇到类似的问题，你现场想恐怕是很难的。下面我就来介绍一下如何用数组加强哈希表，轻松实现 `randomKey()` API。

实现思路

其实我前面给你分析拉链法，就是故意误导你的。和 [链表加强哈希表](#) 一样，只要你陷入到细节里面，那肯定觉得很复杂。

所以说，不要陷入细节。那什么拉链法线性探查法，只是给你介绍下哈希表的运行原理，了解一下为啥它的复杂度是那样。

现在，以及未来做题的时候，你只要记住哈希表是一个能进行键值操作的数据结构，就行了，把它当成一个黑盒，不要去管它的底层实现。

紧凑的数组可以随机返回一个元素，现在我们想随机返回哈希表的一个键，那么最简单的方法就是这样：

```
// 伪码思路
class MyArrayHashMap {
    // arr 数组存储哈希表中所有的 key
    vector<int> arr;
    unordered_map<int, int> map;

    // 添加/修改 key-value 对，时间复杂度  $O(1)$ 
    void put(int key, int value) {
        if (!map.count(key)) {
            // 新增的 key 加入到 arr 数组中
            arr.push_back(key);
        }
        map[key] = value;
    }
}
```

```

    }

    // 获取 key 对应的 value, 时间复杂度 O(1)
    int get(int key) {
        return map[key];
    }

    // 新增 API: 随机返回一个 key, 要求时间复杂度 O(1)
    int randomKey() {
        // 生成 [0, arr.size()) 的随机索引
        return arr[rand() % arr.size()];
    }

    // 删除 key-value 对, 时间复杂度 O(1)
    void remove(int key) {
        ...
    }
};

```

这个思路非常简单，就是用 一个数组 `arr` 维护哈希表中所有的键，然后通过随机索引返回一个键。这样就能保证均匀随机，且时间复杂度是 $O(1)$ 。

但你注意，我没有实现哈希表的 `remove` 方法。因为这个方法不仅要删除哈希表 `map` 中的 `key`，还要删除 `arr` 数组中的元素 `key`，而删除数组中的元素时间复杂度是 $O(N)$ ，因为我们需要搬移数据以保持元素的连续性。

那么有没有办法让 `arr` 数组不用搬移数据，还能保持元素的连续性呢？

$O(1)$ 时间删除数组中的任意元素

其实可以做到：你可以把待删除的元素，先交换到数组尾部，然后再删除，数组尾部删除元素的时间复杂度是 $O(1)$ 。

当然，这样的代价就是数组中的元素顺序会被打乱，但是对于我们当前的场景来说，数组中的元素顺序并不重要，所以打乱了也无所谓。

比如 `arr = [1, 2, 3, 4, 5]`，如果要删除 2，我先把 2 交换到数组尾部，变成 `arr = [1, 5, 3, 4, 2]`，然后只需花 $O(1)$ 的时间即可删除尾部元素 2，且数组的连续性不会被破坏。

是不是思路一下就打开了？

但还有个问题，上面的例子中，你想把元素 2 交换到数组尾部，首先你得知道 2 在数组中的索引是 1，然后才能通过索引进行交换。

如何知道一个元素在数组中对应的索引呢？正常来说需要遍历数组寻找目标元素，这样时间复杂度是 $O(N)$ 。

但是现在不是有哈希表么，键值映射是干啥的？不就是帮你优化这种需要傻乎乎遍历的场景的么？

也就是说，你可以在哈希表中建立**数组元素到数组索引的映射关系**，这样就能在 $O(1)$ 的时间复杂度内找到数组元素对应的索引了。

好了，讲到这里，整个思路已经比较清晰，下面直接看代码实现吧。

代码实现

```
#include <iostream>
#include <random>
#include <vector>
#include <unordered_map>

using namespace std;

template<typename K, typename V>
class MyArrayHashMap {
    struct Node {
        K key;
        V val;

        Node(K key, V val) : key(key), val(val) {
        }
    };

    // 存储 key 和 key 在 arr 中的索引
    unordered_map<K, int> map;

    // 真正存储 key-value 的数组
    vector<Node> arr;

    // 随机数生成器
    default_random_engine e;

public:

    MyArrayHashMap() {
        e.seed(random_device{}());
    }

    V get(K key) {
        if (!map.count(key)) {
            return NULL;
        }
        int index = map[key];
        return arr[index].val;
    }
};
```

```

void put(K key, V val) {
    if (containsKey(key)) {
        // 修改
        int i = map[key];
        arr[i].val = val;
        return;
    }

    // 新增
    arr.push_back(Node(key, val));
    map[key] = arr.size() - 1;
}

void remove(K key) {
    if (!map.count(key)) {
        return;
    }
    int index = map[key];
    Node node = arr[index];

    // 1. 最后一个元素 e 和第 index 个元素 node 换位置
    Node e = arr.back();
    swap(arr[index], arr.back());

    // 2. 修改 map 中 e.key 对应的索引
    map[e.key] = index;

    // 3. 在数组中删除最后一个元素
    arr.pop_back();

    // 4. 在 map 中删除 node.key
    map.erase(node.key);
}

// 随机弹出一个键
K randomKey() {
    int n = arr.size();
    uniform_int_distribution<int> u(0, n - 1);
    int randomIndex = u(e);
    return arr[randomIndex].key;
}

bool containsKey(K key) {
    return map.count(key);
}

int size() {
    return map.size();
}

```

```
};

int main() {
    MyArrayHashMap<int, int> map;
    map.put(1, 1);
    map.put(2, 2);
    map.put(3, 3);
    map.put(4, 4);
    map.put(5, 5);

    cout << map.get(1) << endl; // 1
    cout << map.randomKey() << endl;

    map.remove(4);
    cout << map.randomKey() << endl;
    cout << map.randomKey() << endl;

    return 0;
}
```

到这里，ArrayHashMap 结构就实现完了。如果你想实现 ArrayHashSet，只需要简单封装 ArrayHashMap 即可，我这里就不写代码了。

布隆过滤器

一句话总结

布隆过滤器的核心能力是：

- 在超大规模的数据集中，仅使用少量内存空间，即可快速判断一个元素是否存在。
- 具备数据隐私性。即，可以在不暴露具体数据的情况下，判断一个元素是否存在。

它的核心原理是，不存储具体数据，而仅仅存储数据指纹（哈希值），通过比对指纹来判断数据是否存在。

位图原理及实现

一句话总结

位图（BitMap）是一种非常节省空间的数据结构，它用一个比特位（bit）的 0 和 1 来标记某个元素是否存在。

在后面做算法题时，我们会经常用到类似 `boolean[] visited` 这样的布尔数组，来记录数组中那些元素已经被访问过。

```
// 假设 nums 是一个包含 1000 个整数的数组
int[] nums = {...}

// 我们在写算法时
```

```
// 可能会用一个布尔数组来记录 nums 中那些元素已经被访问过
boolean[] visited = new boolean[nums.length];
visited[10] = true;
visited[100] = true;
```

我们来仔细观察这个场景，是否存在优化空间？

布尔类型只有 `true` 和 `false` 两种状态，理论上只需要 1 个比特位 (bit) 的 0 和 1 就可以表示。

但在大部分编程语言中，由于内存寻址等原因，一个布尔元素通常会占用 1 字节 (byte)，也就是 8 个比特位的内存。

这就意味着，编程语言内置的布尔数组 `boolean[]` 实际上浪费了 7/8 的内存空间。

那么我们是否可以优化？答案是肯定的。

提示

在实际开发和求解算法题的过程中，我们使用编程语言提供的布尔数组就够了，**除非需要处理的数据规模非常大，否则没必要为了节省这一点内存空间而引入位图这种结构。**

比如后文介绍的 [布隆过滤器](#)，专门为了处理超大规模数据而设计，才需要使用位图这种结构进行优化。

优化内存的思路很简单，就是要充分利用每个比特位。

我们可以用 `long[] words` 来作为底层存储。不过索引的计算规则需要稍微改造一下，以便我们精确地控制每个比特位。

一个 `long` 类型数据是 8 byte，也就是 64 bit，假设我们想访问第 135 个比特位，那么：

1. $135 / 64 = 2$ ，所以目标比特位在 `words` 数组的索引 2 的位置，即 `words[2]`。
2. $135 \% 64 = 7$ ，所以目标比特位在 `words[2]` 元素的第 7 个比特位（从 0 开始数）。

如果你绕不过来，可以想象 `long[] words` 是一个二维数组，`words[i]` 就是访问二维数组的第 *i* 行，每一行有 64 个比特位。你想访问第 *k* 个比特位，就是访问二维数组的第 $k / 64$ 行，第 $k \% 64$ 列。

了解了索引转化规则之后就简单了，我们暴露几个方法，让用户操作位图：

- `boolean get(int bitIndex)`：判断第 `bitIndex` 个比特位是否为 1。
- `void set(int bitIndex)`：将第 `bitIndex` 个比特位设置为 1。
- `void clear(int bitIndex)`：将第 `bitIndex` 个比特位设置为 0。

```
#include <iostream>
#include <vector>
#include <stdexcept>
using namespace std;
```



```

class MyBitSet {
private:
    // 使用 unsigned long 数组作为位图的底层存储
    vector<unsigned long> words;
    // 位图能够存储的最大元素值 + 1
    int size;

public:
    MyBitSet(int size) : size(size) {
        // 根据 size 计算需要多少个 unsigned long long 来存储
        int arraySize = size / 64 + 1;
        words.resize(arraySize, 0);
    }

    // 判断指定比特位是否为 1
    bool get(int bitIndex) {
        if (bitIndex < 0 || bitIndex >= size) {
            throw out_of_range("bitIndex must be between 0 and " +
to_string(size - 1));
        }
        // 找到 bitIndex 在 words 数组中的索引
        int wordIndex = bitIndex / 64;
        // 找到 bitIndex 在 unsigned long long 值中的具体 bit 位
        int bitOffset = bitIndex % 64;
        // 使用 & 操作判断该位是否为 1
        return (words[wordIndex] & (1ULL << bitOffset)) != 0;
    }

    // 将指定比特位设置为 1
    void set(int bitIndex) {
        if (bitIndex < 0 || bitIndex >= size) {
            throw out_of_range("bitIndex must be between 0 and " +
to_string(size - 1));
        }
        // 找到 bitIndex 在 words 数组中的索引
        int wordIndex = bitIndex / 64;
        // 找到 bitIndex 在 unsigned long long 值中的具体 bit 位
        int bitOffset = bitIndex % 64;
        // 使用 | 操作将该位置 1
        words[wordIndex] |= (1ULL << bitOffset);
    }

    // 将指定比特位设置为 0
    void clear(int bitIndex) {
        if (bitIndex < 0 || bitIndex >= size) {
            throw out_of_range("bitIndex must be between 0 and " +
to_string(size - 1));
        }
        int wordIndex = bitIndex / 64;

```

```

        int bitOffset = bitIndex % 64;
        // 使用 & 和 ~ 操作将该位置 0
        words[wordIndex] &= ~(1ULL << bitOffset);
    }
};

int main() {
    MyBitSet bitSet(1000);

    bitSet.set(10);
    bitSet.set(100);
    bitSet.set(500);

    cout << "Get 10: " << (bitSet.get(10) ? "true" : "false") << endl;
    // true
    cout << "Get 100: " << (bitSet.get(100) ? "true" : "false") << endl;
    // true
    cout << "Get 200: " << (bitSet.get(200) ? "true" : "false") << endl;
    // false

    bitSet.clear(100);
    cout << "Get 100 after clear: " << (bitSet.get(100) ? "true" : "false")
    << endl; // false

    return 0;
}

```

上面的代码就实现了位图的核心功能。不过其中的运算可以有一些小优化：

- `bitIndex / 64` 可以优化为 `bitIndex >> 6`，因为 $2^6=64$ 。
- `bitIndex % 64` 可以优化为 `bitIndex & 63`，因为当除数是 2 的整数次幂（比如 $64=2^6$ ）时，取模运算 `% 64` 等价于位运算 `& (64 - 1)`。

位运算的效率比普通运算的效率 high 得多，所以优化后的代码性能会更好一些。这几个位运算技巧你可以先记住，原理会在 [常用的位操作](#) 中介绍，这里不展开讲了。

布隆过滤器原理及实现

典型的超大规模数据场景有：

- 判断一个 HTTP 请求的 URL 是否在恶意 URL 列表中。这个列表的数量可以达到上亿，我们可以借助布隆过滤器，保证这个查询仅消耗少量的内存和查询时间，否则会影响用户体验。
- 在大数据存储系统中，海量的数据一般会分散存储在多个不同的文件中。查询一个数据时，需要在磁盘上遍历多个文件才能找到，效率很差。我们可以为每个文件在内存中维护一个布隆过滤器，以便快速判断目标数据是否存在于该文件中，避免无效的磁盘 IO 操作。

说到判断元素是否存在，首先应该想到前文讲的 [哈希集合](#)，它可以在 $O(1)$ 的时间增删元素，可以在 $O(1)$ 的时间判断一个元素是否存在。

但如果数据规模特别大，那就不行了。

因为哈希集合本质上就是哈希表，而在 [哈希表的代码实现](#) 中，我们必须要用一个 `KVNode` 类把键值数据存储在内存中，以便处理哈希冲突。

所以哈希集合的空间复杂度是 $O(N)$ ，随着存储元素的数量增加，占用的内存也会线性增加。在超大数据规模场景下，有限的内存肯定是无法加载所有数据的。

同时，因为哈希集合要存储实际的数据，也就不具备数据隐私性。

那么，布隆过滤器是如何实现的呢？有了布隆过滤器，是否还需要哈希集合呢？

布隆过滤器的核心就是一个 [位图](#) 和 k 个不同的哈希函数。

当我们想往布隆过滤器中添加一个元素时，我们用 k 个哈希函数对这个元素进行哈希计算，得到 k 个不同的哈希值，这些哈希值就是位图的索引。然后我们把位图中的这几个索引对应的位都设置成 1。

当我们想判断一个元素是否存在于布隆过滤器时，我们也是用 k 个哈希函数对这个元素进行哈希计算，得到 k 个索引。

然后我们去看位图中的这 k 个索引对应的位是否**全部**为 1。如果其中有一个位不为 1，那么这个元素**一定不存在**于集合中。如果这 k 个位都为 1，那么这个元素**可能存在**于集合中。

为什么是**可能存在**呢？

因为随着布隆过滤器中存入的元素越来越多，位图中的这几个索引可能恰巧被其他元素设置成了 1，这就是所谓的**假阳性误判**（false positive）。

下面，我们复用 [位图的原理与实现](#) 中的 `MyBitSet` 类，来实现一个简化的布隆过滤器代码，展示其核心原理：

```
#include <iostream>
#include <string>
#include <cmath>
#include <functional>
using namespace std;

template<typename T>
class SimpleBloomFilter {
private:
    // 位图的大小
    int bitSetSize;
    // 位图
    MyBitSet* bitSet;
    // 哈希函数的个数
```

```

int k;

// 模拟多个哈希函数，实际生产环境中应该使用更复杂的哈希算法
int hash(const T& element, int seed) {
    // 这里简化处理，用内置的 hash 函数和递增的索引作为种子来模拟多个哈希函数
    // 在实际应用中，为了减少哈希冲突，应该使用更复杂的哈希函数
    // 同时，种子也应该选择无规律的大质数，而不是简单的递增索引
    std::hash<T> hasher;
    size_t h = hasher(element);
    return abs(static_cast<int>(h) + seed) % bitSetSize;
}

public:
// 构造一个布隆过滤器，指定位图大小和哈希函数个数
SimpleBloomFilter(int bitSetSize, int hashFunctionNum)
    : bitSetSize(bitSetSize), k(hashFunctionNum) {
    bitSet = new MyBitSet(bitSetSize);
}

~SimpleBloomFilter() {
    delete bitSet;
}

// 添加元素
void add(const T& element) {
    // 获取 k 个不同的哈希值
    // 将这 k 个哈希值对应的位图中的位都设置为 1
    for (int i = 0; i < k; i++) {
        int hashValue = hash(element, i);
        bitSet->set(hashValue);
    }
}

// 判断元素是否存在
bool contains(const T& element) {
    // 获取 k 个不同的哈希值
    // 检查这 k 个哈希值对应的位图中的位是否全部为 1
    for (int i = 0; i < k; i++) {
        int hashValue = hash(element, i);
        if (!bitSet->get(hashValue)) {
            return false;
        }
    }
    return true;
}

};

int main() {
    // 创建一个位数组大小为 1000000，使用 3 个哈希函数的布隆过滤器
    SimpleBloomFilter<string> bloomFilter(1000000, 3);
}

```

```

// 添加元素
bloomFilter.add("apple");
bloomFilter.add("banana");
bloomFilter.add("orange");

// 检查元素是否存在
cout << "Contains apple: " << (bloomFilter.contains("apple") ? "true" :
"false") << endl;      // true
cout << "Contains banana: " << (bloomFilter.contains("banana") ? "true"
: "false") << endl;    // true
cout << "Contains grape: " << (bloomFilter.contains("grape") ? "true" :
"false") << endl;     // false

return 0;
}

```

这段代码实现了一个简化的布隆过滤器，但是其中还有很多地方可以进一步优化：

- **哈希函数过于简陋**：示例代码中的 `hash` 函数通过 `element.hashCode() + seed` 的方式来模拟多个哈希函数，这种方式生成的哈希值分布不够均匀，会增加哈希冲突的概率，而导致实际的误判率高于预期。在生产环境中，应该使用更可靠的哈希算法，例如 [MurmurHash](#) 或 [FNV hash](#) 等。常用的技巧有两种：
 1. **双重哈希 (Double Hashing)**：使用两个高质量的哈希函数来模拟 k 个哈希函数：

$$h_i = (h_1 + i \cdot h_2) \bmod m$$
 2. **随机种子 (Random Seeds)**：预先生成 k 个不同的随机种子（通常选择大的素数），然后用同一个哈希函数和这 k 个种子计算出 k 个不同的哈希值。这比示例代码中简单地使用递增索引 $0, 1, 2, \dots$ 作为种子要好得多，因为它能更好地模拟独立的哈希函数，进一步减少冲突。
- **缺少参数的自动优化**：布隆过滤器的位图大小和哈希函数个数需要根据预估的数据量和可容忍的误判率来计算得出，这样才能最高效地利用内存。示例代码中这两个参数是写死的，这在实际应用中是不够灵活的。一个更健壮的实现应该提供一个构造函数，接收 n （预期插入的元素数量）和 fpp （可接受的误判率）作为参数，然后根据以下公式自动计算出最优的位图大小 `bitSetSize` 和哈希函数个数 k ：

$$k = \frac{\ln(2) \cdot \ln(1/fpp)}{\ln(2)^2} \cdot \ln(2) \cdot n$$

布隆过滤器 vs 哈希集合

布隆过滤器和哈希集合都可以用来判断一个元素是否存在于一个集合中，但它们在实现原理、性能和适用场景上存在显著差异。

特性	布隆过滤器	哈希集合
存储内容	只存储数据的「指纹」（哈希值），不存储数据本身。	存储数据本身，例如 <code>KVNode</code> 对象。
内存占用	极低，空间复杂度为 $O(\text{bitSetSize})$ ，空间占用不随元素本身的大小变化。	较高，空间复杂度为 $O(N)$ ，其中 N 是元素个数，并且与元素本身的大小有关。

特性	布隆过滤器	哈希集合
查询结果	存在误判率（False Positive）：可能将不存在的元素判断为存在。	绝对精确，没有误判。
删除操作	标准的位图实现不支持删除。删除一个元素的位标记可能会影响其他元素。	支持删除操作。
数据隐私	具备数据隐私性，因为无法通过布隆过滤器反推出原始数据。	不具备数据隐私性，因为存储了原始数据。

核心权衡点：

布隆过滤器的核心是一种**权衡（trade-off）** 思想，它用**较低的内存占用**和**一定的误判率**换取了**极高的查询效率**。

如果你可以容忍误判率，那么布隆过滤器是一个非常高效的选择。

比如垃圾邮件过滤的场景，相比布隆过滤器带来的性能提升，偶尔把一个垃圾邮件误判为正常邮件是可以接受的。

再比如，对于开头讲的文件查询的场景，即便出现误判，代价只是多读取一个文件，如果误判率足够低，这种情况带来的性能损失是可以接受的。

（假阳性误判：存在的不一定有，没有的一定不存在。）